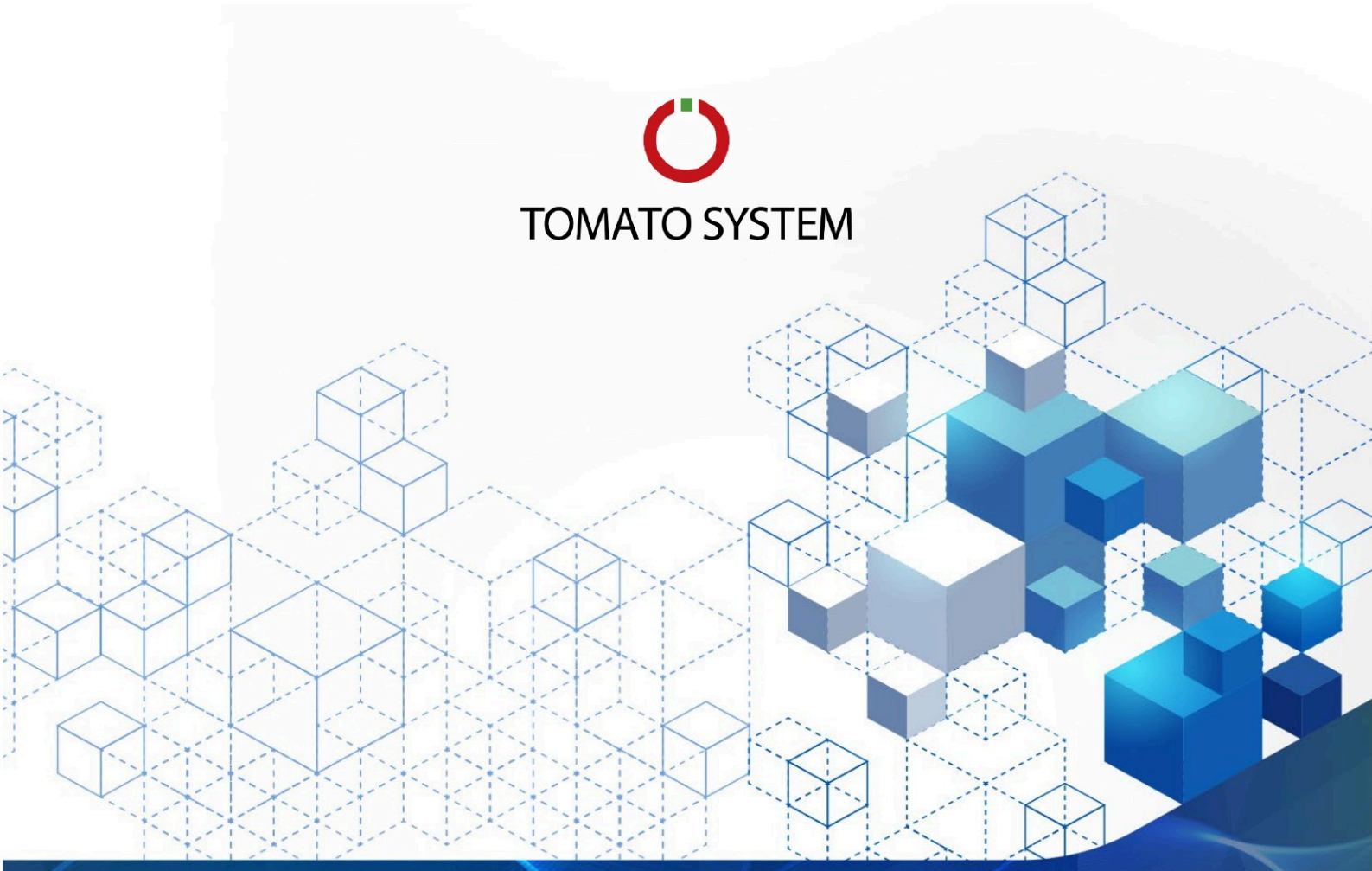
TOMATO SYSTEM

eXBuilder6

퍼블리싱 가이드



Copyright(c) TOMATO SYSTEM. All Right Reserved.

[문서 이력]

버전	주요 제/개정 내용	작성일	작성자	승인자	시행일
1.0	최초 작성	2026-11-06	한예인		

개요

eXBuilder6는 토마토시스템에서 개발한 UI/UX 개발 플랫폼으로, 이클립스 기반의 위지윅(WYSIWYG) 환경과 표준 웹 기술을 결합하여 고성능 엔터프라이즈 UI를 구축하는 도구입니다.

소개

이 가이드는 디자인 시스템의 일관성 유지, 컴포넌트 재사용성 극대화, 퍼블리싱 표준 제시 등의 내용을 포함하고 있습니다.

브라우저 지원 범위

퍼블리싱 결과물은 eXBuilder6가 지원하는 최신 웹 표준 브라우저 환경을 기준으로 작성되었으며, 아래 브라우저에서 정상적으로 작동하도록 검수합니다.

구분	작업 버전
Chrome	150.0.7871.125
Edge	150.0.4078.65

- **기준 버전:** 위 버전은 검수 기준이며, 이후 출시되는 안정(Stable) 버전에서도 정상 작동을 목표로 합니다. 프로젝트에서 별도의 기준이 확정되면 해당 기준을 우선 적용합니다.

기술 안내

네이밍 규칙

- 모든 스타일 클래스는 소문자와 하이픈(-)을 사용하는 케밥 케이스(kebab-case)를 원칙으로 합니다. (예: .badge.outline-primary, .cl-inputbox.is-error)
- eXBuilder6 컨트롤에 대한 퍼블리싱은 해당 컨트롤의 루트 요소에 스타일 클래스를 부여하여 하위 노드들을 제어하는 방식을 지향합니다. 이는 인라인 스타일을 최소화하고 툴의 업데이트에 유연하게 대응하기 위함입니다.

폴더 구조

모든 스타일 관련 파일은 프로젝트 루트의 clx-src/theme/ 경로에서 통합 관리됩니다.

경로	파일명 / 목적	설명
theme/less	cleopatra.less	프로젝트의 최종 엔트리 파일. 모든 스타일을 병합하여 컴파일함
theme/less	high-contrast.less	고대비 테마의 최종 엔트리 파일
theme/less/common	common.part.less	디자인 토큰 및 공통 믹스인을 @import 하는 브릿지 파일
theme/less/component	*.part.less	기능/컨트롤별 독립된 스타일 파일

주의사항

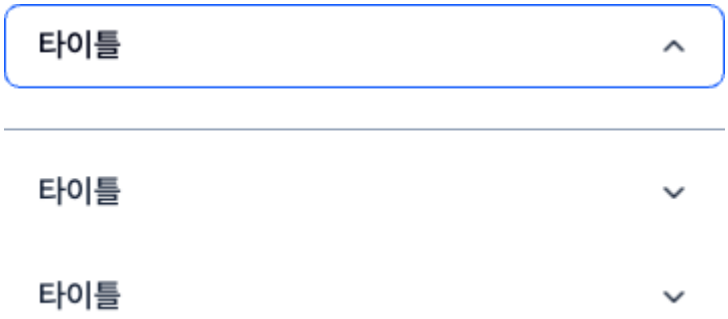
- **브라우저 업데이트 관련:** 본 가이드는 검수 당시의 브라우저 버전을 기준으로 제작되었습니다. 배포 이후 브라우저 엔진의 자동 업데이트나 정책 변경으로 인해 발생하는 레이아웃 왜곡 및 기능 오동작에 대해서는 퍼블리싱 팀의 귀책사유가 아님을 명시합니다.
- **임의 수정 금지:** 가이드에서 제시한 표준 클래스 및 구조를 사용자가 임의로 변경(Inline Style 강제 적용, 스크립트를 통한 DOM 조작 등)하여 발생하는 모든 시각적 결함 및 오류는 사용자(개발자)의 책임으로 간주합니다.
- **제3자 리소스 라이선스:** 가이드에 포함된 폰트, 아이콘 등 외부 리소스는 해당 라이선스 규정을 따르며, 허가되지 않은 상업적 재배포나 용도 외 사용으로 발생하는 저작권 분쟁에 대해 본 가이드는 책임을 지지 않습니다.
- **실제 데이터와의 차이:** 가이드 내 예시 화면은 퍼블리싱 샘플 데이터 기준이며, 실제 DB 데이터가 바인딩되는 시점의 데이터 길이, 특수문자 포함 여부 등에 따라 디자인이 일부 상이할 수 있습니다.

추가 도움말 및 리소스

- **eXBuilder6 기술지원:** https://tech.tomatosystem.co.kr/

아코디언

슬라이드로 컨트롤들의 집합이나 컨트롤을 보여주는 컨트롤



구조

아코디언의 구조는 다음과 같다.

아코디언

```
<CONTROL class="cl-control cl-accordion">
  <HEADER class="cl-accordion-header"><TEXT class="cl-text" />
    <EXPANDER class="cl-accordion-expander" />
  </HEADER>
  <SECTION class="cl-accordion-section">
    <CONTENT class="cl-accordion-content" />
  </SECTION>
</CONTROL>
```

ⓘ 테마 파일

accordion.part.less

사용 가이드라인

ⓘ 섹션 아이템의 컨테이너 레이아웃

컨텐츠에 맞는 레이아웃 유형을 사용하며, 간격과 여백은 콘텐츠 유형에 따라 달라질 수 있다.

배지

컴포넌트에 대한 빠른 인지와 탐색을 돕기 위해 컴포넌트 근처에 표시되는 작은 문자 또는 숫자 데이터



구조

배지는 아웃풋 컨트롤의 구조와 같다.

아웃풋

```
<CONTROL class="cl-control cl-output badge"><TEXT class="cl-text" /></CONTROL>
```

ⓘ 테마 파일

badge.part.less

사용 가이드라인

- **컨트롤의 너비와 높이:** 컨트롤의 너비와 높이는 데이터의 길이에 맞게 상황에 따라 유동적으로 적용하기 위해 자동 크기 관련 속성 지정
- **디스플레이 표현식:** 표시되는 텍스트를 다양한 값으로 혼합하여 보여주어야 하는 경우 displayExp 속성 적용

확장



등급

옅은 톤(soft)의 색상 배지와 순위·등급 배지를 제공한다.



웹 접근성 유의사항

- 접근성과 명확한 이해를 위해 항상 텍스트 레이블을 제공

버튼

마우스 클릭이 가능한 컨트롤



구조

버튼 컨트롤의 구조는 다음과 같다.

버튼

```
<CONTROL class="cl-control cl-button">
  <WRAP class="cl-text-wrapper">
    <ICON class="cl-icon" /><TEXT class="cl-text" />
  </WRAP>
</CONTROL>
```

ⓘ 테마 파일

button.part.less

사용 가이드라인

- **컨트롤의 너비 및 높이:** 컨트롤의 너비는 데이터의 길이에 맞게 상황에 따라 유동적으로 적용, 높이는 24px적용
- **아이콘 표시:** icon 속성 설정
  - 아이콘의 방향은 iconAlign 속성으로 적용

확장

계층

버튼과 관련된 사용자 과업, 플로우의 중요도에 따라 버튼 간 위계를 명확하게 구분하여 사용

- 서비스의 주요 색상을 배경색으로 채운 버튼은 강조 버튼으로 사용자로부터 중요한 행동을 유도하고자 할 때 사용
- 윤곽선 버튼은 기본 버튼으로 중간 수준의 강조가 필요한 경우에 사용. 버튼과 관련된 행동의 중요도가 낮거나 한 페이지 또는 한 개의 뷰에서 중요도가 동일한 여러 개의 기능이 제공되어야 하는 경우에 적합
- 텍스트 버튼은 가장 낮은 수준의 강조를 가지며 다양한 기능이 제공되어야 할 때 사용하기 적합
- 인라인 버튼은 폼 또는 그리드 내에서 다양한 기능이 제공되어야 할 때 사용하기 적합



LESS

```less
// 강조 버튼
.cl-button.btn-primary {}

// 윤곽선(기본) 버튼
.cl-button.btn-base {}

// 텍스트 버튼
.cl-button.btn-text {}

// 인라인 버튼
.cl-button.btn-inline {}
```

JS

```js
// 강조 버튼
Button.style.addClass('btn-primary')

// 윤곽선(기본) 버튼
Button.style.addClass('btn-base')

// 텍스트 버튼
Button.style.addClass('btn-text')

// 인라인 버튼
Button.style.addClass('btn-inline')
```

버튼 메인형

메인/서브는 주요 흐름의 의사결정 액션에, 조회는 화면 데이터를 갱신하는 화면 제어 액션에 사용. 조회조건 영역의 버튼은 기본 h36 그대로, 화면 타이틀 영역의 버튼은 class="large"를 같이 지정해 h40으로. 디폴트/호버/포커스/눌림 상태는 실제로 마우스를 올려 확인



LESS

```less
// 메인(강조)
.cl-button.btn-primary {}

// 서브
.cl-button.btn-base {}

// 조회
.cl-button.btn-primary-outline {}

// 큰 사이즈(h40) 조합
.cl-button.btn-primary.large {}
```

JS

```js
// 메인(강조)
Button.style.addClass('btn-primary')

// 서브
Button.style.addClass('btn-base')

// 조회
Button.style.addClass('btn-primary-outline')

// 큰 사이즈(h40) 조합
Button.style.addClass('large')
```

버튼 기능형

그리드 툴바 등에서 반복 노출되는 보조 액션 버튼. h36 규격으로 사용



LESS

```less
// 기능
.cl-button.btn-function {}

// 행추가 / 저장(강조-옅음)
.cl-button.btn-primary-soft {}

// 행삭제(위험-옅음)
.cl-button.btn-danger-soft {}
```

JS

```js
// 기능
Button.style.addClass('btn-function')

// 행추가 / 저장(강조-옅음)
Button.style.addClass('btn-primary-soft')

// 행삭제(위험-옅음)
Button.style.addClass('btn-danger-soft')
```

아이콘 버튼

아이콘과 텍스트가 함께 사용되는 버튼

`icon-{name}` 형태의 아이콘 클래스를 버튼 class에 함께 지정



LESS

```less
.cl-button.icon- {
  name
}

.cl-icon {}
```

JS

```js
Button.style.addClass('icon-{name}');
```

아이콘 목록

> ⓘ 테마 파일
>
> theme/less/component/button.part.less

클래스명	예시
icon-excel	엑셀
icon-arrow-down	접기
icon-arrow-up	펼치기
icon-right	추가
icon-left	제거

아이콘만 있는 버튼

클래스명	예시
icon-link	
icon-search	
icon-input-search	



LESS

```less
// 아이콘만 있는 버튼
.cl-button.icon {}
```

JS

```js
// 아이콘만 있는 버튼
Button.style.addClass('icon');
```

텍스트 버튼

LESS

```less
.cl-button.btn-text {}
```

JS

```js
Button.style.addClass('btn-text');
```

링크 버튼

LESS

```less
// 링크버튼(일반)
.cl-button.btn-link {}
```

JS

```js
// 링크버튼(일반)
Button.style.addClass('btn-link');
```

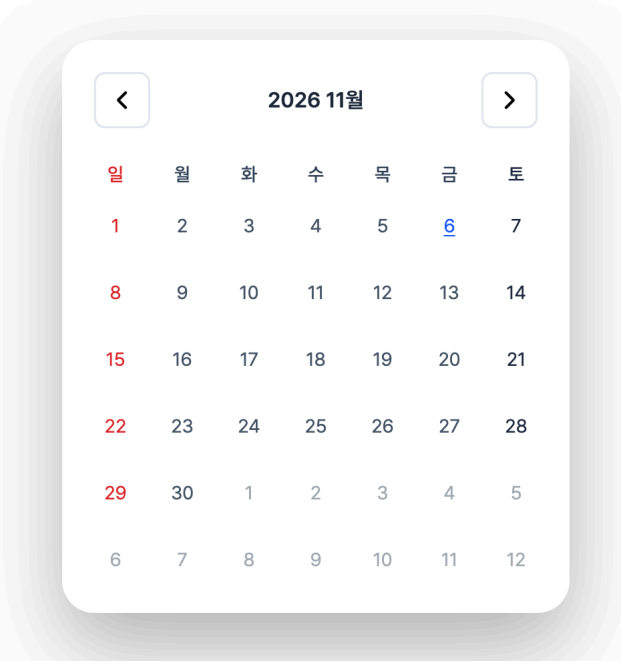
웹 접근성 유의사항

- 링크로 작동하는 버튼은 `ariaButtonType = 'link'` 로 설정한다.
- 버튼의 기능을 설명할 수 있는 텍스트 레이블이 있는 아이콘 버튼에 대체 텍스트를 제공하지 않는다.

- 버튼의 접근 가능한 이름은 버튼을 통해 실행되는 기능을 적절하게 설명할 수 있는 내용으로 제공
  - 텍스트 레이블 없이 아이콘만 표시되는 버튼은 텍스트 이름이 이미지에 대한 설명이 아니라 버튼의 기능에 대한 정확한 설명을 제공
  - 텍스트 레이블 없이 아이콘만 표시되는 버튼은 `displayExp` 속성을 적용하여 대체 텍스트 제공 (예: `sstr(text, ["cl-sound-only"])`)

캘린더

날짜를 입력하는 컨트롤



구조

캘린더의 구조는 다음과 같다.

캘린더(일)

```
<CONTROL class="cl-control cl-calendar cl-calendar-day">
  <FIELD class="cl-calendar-field">
    <WRAP class="cl-calendar-header-wrapper">
      <HEADER class="cl-calendar-header">
        <BUTTON class="cl-calendar-header-before" />
        <BUTTON class="cl-calendar-header-prev" />
        <TEXT class="cl-calendar-header-text" />
        <BUTTON class="cl-calendar-header-next" />
        <BUTTON class="cl-calendar-header-after" />
      </HEADER>
    </WRAP>
    <WRAP class="cl-calendar-content-wrapper">
      <WRAP class="cl-calendar-content">
        <>
          <HEADER class="cl-calendar-content-header cl-calendar-content-sun" />
          <HEADER class="cl-calendar-content-header cl-calendar-content-mon" />
          <HEADER class="cl-calendar-content-header cl-calendar-content-tue" />
          <HEADER class="cl-calendar-content-header cl-calendar-content-wed" />
          <HEADER class="cl-calendar-content-header cl-calendar-content-thu" />
          <HEADER class="cl-calendar-content-header cl-calendar-content-fri" />
          <HEADER class="cl-calendar-content-header cl-calendar-content-sat" />
        </>
        <>
          <DAY class="cl-other-month cl-calendar-content-mon"><TEXT class="cl-text" /></DAY>
          <DAY class="cl-other-month cl-calendar-content-tue"><TEXT class="cl-text" /></DAY>
          <DAY class="cl-content-day cl-calendar-content-wed"><TEXT class="cl-text" /></DAY>
          ...
          <DAY class="cl-content-day cl-calendar-content-wed"><TEXT class="cl-text" /></DAY>
          <DAY class="cl-other-month cl-calendar-content-thu"><TEXT class="cl-text" /></DAY>
        </>
      </WRAP>
    </WRAP>
    <WRAP class="cl-calendar-footer-wrapper">
      <FOOTER class="cl-calendar-footer">
        <TEXT class="cl-calendar-footer-text" />
      </FOOTER>
```

캘린더(년/월)

```
// 년 달력
<CONTROL class="cl-control cl-calendar cl-calendar-year">
  <FIELD class="cl-calendar-field">
    <WRAP class="cl-calendar-header-wrapper">
      <HEADER class="cl-calendar-header">
        <BUTTON class="cl-calendar-header-prev" />
        <TEXT class="cl-calendar-header-text" />
        <BUTTON class="cl-calendar-header-next" />
      </HEADER>
    </WRAP>
    <WRAP class="cl-calendar-content-wrapper">
      <WRAP class="cl-calendar-content">
        <YEAR class="cl-other-month" />
        <YEAR class="cl-calendar-content-year" />
        ...
        <YEAR class="cl-calendar-content-year" />
        <YEAR class="cl-other-month" />
      </WRAP>
    </WRAP>
    <WRAP class="cl-calendar-footer-wrapper">
      <FOOTER class="cl-calendar-footer">
        <TEXT class="cl-calendar-footer-text" />
      </FOOTER>
    </WRAP>
  </FIELD>
</CONTROL>

// 월 달력
<CONTROL class="cl-control cl-calendar cl-calendar-month">
  <FIELD class="cl-calendar-field">
    <WRAP class="cl-calendar-header-wrapper">
      <HEADER class="cl-calendar-header">
        <BUTTON class="cl-calendar-header-prev" />
        <TEXT class="cl-calendar-header-text" />
        <BUTTON class="cl-calendar-header-next" />
      </HEADER>
```

ⓘ 테마 파일

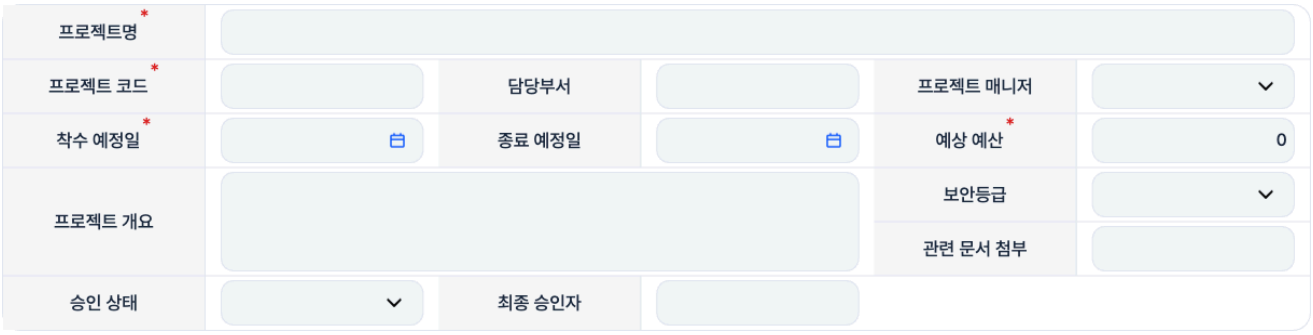
calendar.part.less

사용 가이드라인

- **날짜 형식 설정:** `calendarType` 속성 값 변경

폼

데이터를 하나 이상의 행과 열로 조직화하여 표현하는 형식



LESS

```less
// 폼
.cl-container.form-base {

  // 레이블
  .label {
    &.required {}
  }
}
```

JS

```js
// 폼
Container.style.addClass('form');

// 레이블
Output.style.setClasses(['label', 'required']);

// 입력 필드
Inputbox.style.addClass('required');
```

속성	타입	설정값
margins	string	'4px 14px'
horizontalSpacing	number	28
verticalSpacing	number	8
horizontalSeparatorType	string	'BY_CLASS'
horizontalSeparatorWidth	number	1
verticalSeparatorType	string	NONE
verticalSeparatorWidth	number	0
scrollable	boolean	false

구조

폼 레이아웃이 적용된 그룹의 구조는 다음과 같다.

그룹(폼 레이아웃)

```
<CONTROL class="cl-control cl-container">
  <WRAP class="cl-layout">
    <WRAP class="cl-layout-content">
      <CONTROL class="cl-control cl-output label" />
      <CONTROL class="cl-control cl-inputbox" />
      ...
    </WRAP>
  </WRAP>
</CONTROL>
```

ⓘ 테마 파일

form-layout.part.less

사용 가이드라인

레이블

- **[필수] 레이아웃 설정:** `ignoreLayoutSpacing = true`
- **[필수] 클래스 설정:** `label` (필수입력 필드인 경우 `required` 과 함께 설정)
- 레이블이 배치되는 폼 구획의 너비는 기본 160px 로 설정하며, 텍스트 길이에 따라 유동적으로 변경 가능
  - **구획 설정:** `autoSizing = true`

입력 필드

- **클래스 설정:** 필수입력 필드인 경우 `required` 설정
- 입력 필드가 배치되는 폼 구획의 너비는 기본 1fr 로 설정하며, 텍스트 길이에 따라 유동적으로 변경 가능

확장

리스트형

form-base 과 tbl 클래스 적용

form-base는 `verticalSeparator` 를 사용하지 않지만, tbl은 `verticalSeparatorType = 'BY_CLASS'`, `verticalSeparatorWidth = 1` 을 함께 설정하여 세로 구분선을 표시

자산번호	자산명	카테고리	제조사	시리얼번호	보증만료일 *
AST-26-001	MacBook Pro 16	노트북	Apple	SN-99283-X	

자산번호	자산명	카테고리	제조사	시리얼번호	보증만료일
AST-26-001	MacBook Pro 16	노트북	Apple	SN-99283-X	2027-01-10

자산번호	자산명	카테고리	제조사	시리얼번호	보증만료일
구분	MacBook Pro 16	노트북	Apple	SN-99283-X	2027-01-10

LESS

```less
// 폼 (리스트형)
.cl-container.form-base.tbl {
  .labelm .cl-output {
    text-align: center;
  }
}
```

JS

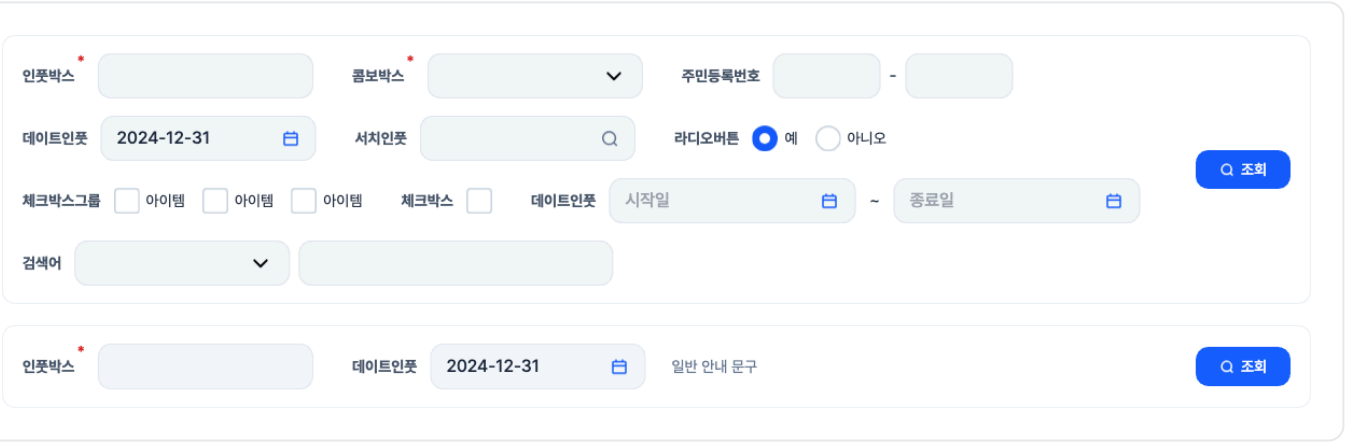
```js
// 폼 (리스트형)
Container.style.setClasses(['form', 'tbl']);

// 레이블
Output.style.setClasses(['label', 'required']);

// 입력 필드
Dateinput.style.addClass('required');
```

조회 영역

search-box 클래스 적용, 조회조건 영역과 조회 버튼 영역(`sch-btn-wrap`)을 하나의 그룹으로 구성



LESS

```less
// 조회 영역
.cl-container.search-box {
  background-color: @color-box-background;
  border: 1px solid @color-box-border;

  // 조회조건 그룹(라벨+필드 묶음) - 그룹 사이 간격은 24px, 라벨-필드 사이만 좁힘
  .cl-container.search-ctrl {
    .cl-layout-wrap:has(.label) {
      margin-right: @gap-6 * -1;
    }
  }

  .label {
    margin-left: @gap-6;
  }

  .cl-container.sch-btn-wrap {
    // 조회 버튼 영역
  }
}
```

JS

```javascript
// 조회 영역
Container.style.setClasses(['search-box']);

// 조회조건 그룹(라벨+필드 묶음)
Container.style.addClass('search-ctrl');

// 조회 버튼 영역
Container.style.addClass('sch-btn-wrap');

// 조회 버튼
Button.style.addClass('btn-search');
```

대상	클래스·ID	필수	설명
조회 영역 그룹	id="grpSearch"	★	조회 영역 루트 그룹의 ID, 여러 개면 숫자를 붙여 구분
조회 영역 그룹	class="search-box"	★	조회 영역 스타일 적용
조회조건 그룹	class="search-ctrl"	★	라벨+필드 쌍을 묶는 그룹, 그룹 사이 간격(24px)보다 라벨-필드 사이 간격을 좁게 보정
조회 버튼 영역 그룹	class="sch-btn-wrap"	★	조회 버튼을 담는 영역, 너비 고정 + 우측 정렬
조회 버튼	class="btn-search"	★	조회 버튼 스타일 적용
복합 입력 그룹	class="form-control"	○	기간·구간 등 필드 2개 이상을 하나로 묶을 때 사용
라벨 / 필드	class="required"	△	필수입력 항목 표시(라벨 * 표시) 및 유효성 검사 대상 지정

사용 가이드라인

- **[필수] 그룹 ID:** `grpSearch` 접두어 사용, 조회 영역이 여러 개인 경우 숫자를 붙여 구분
- **[필수] 클래스 설정:** `search-box`

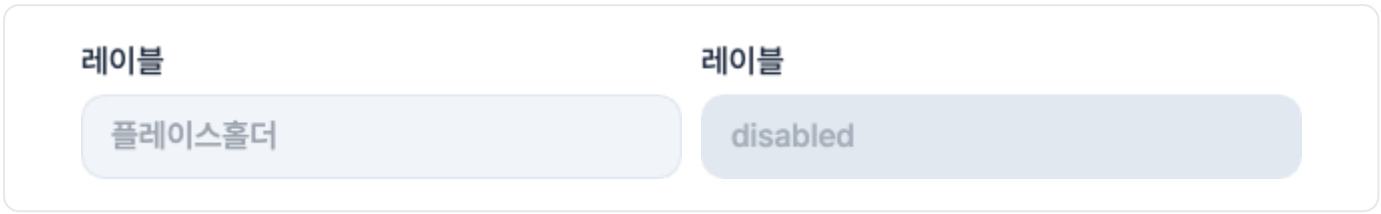
- **구조:** 조회조건 영역과 조회 버튼 영역(`sch-btn-wrap`)을 좌우로 배치
  - 조회조건 영역은 라벨과 필드를 하나의 그룹(search-ctrl)으로 묶어 줄바꿈이 가능한 플로우 레이아웃(flowLayout)으로 배치
  - 각 필드 컨트롤의 flowlayoutdata 에 `allownewline = false` 설정, 줄바꿈이 컨트롤 내부가 아닌 컨트롤 단위로만 일어나도록 함
  - `search-ctrl`은 플로우 간격(24px)을 그룹 사이 여백으로 쓰고, 라벨-필드 사이만 좁아지도록 보정(라벨 margin-left + 라벨이 있는 래퍼의 margin-right 상쇄)
  - 조회 버튼 영역(sch-btn-wrap)은 너비를 고정하고, btn-search 클래스의 조회 버튼을 우측 정렬로 배치
- **배치 순서:** 작업 흐름에 따라 좌측 상단 → 우측 상단 → 좌측 하단 → 우측 하단 순으로 배치(Z 방향)

웹 접근성 유의사항

- 논리적인 순서에 따라 마크업 순서 정렬
  - **표시 순으로 자식 정렬:** CTRL + SHIFT + K 단축키로 기능 사용
  - **아웃라인 뷰 수정:** 컨트롤 배치 순서를 아웃라인 뷰로 확인하고 직접 순서 변경
  - **tabIndex 사용 지양:** `tabIndex` 속성을 직접 지정하지 않도록 주의

입력 필드

글자, 숫자 등 데이터를 입력할 수 있는 컨트롤



구조

인풋 계열 컨트롤의 구조는 다음과 같다.

인풋박스

```
<CONTROL class="cl-control cl-inputbox"><TEXT class="cl-text" />
  <CLEAR class="cl-inputbox-clear" />
</CONTROL>
```

서치인풋

```
<CONTROL class="cl-control cl-searchinput"><TEXT class="cl-text" />
  <CLEAR class="cl-searchinput-clear" />
  <SEARCH class="cl-searchinput-search" />
</CONTROL>
```

마스크에디터

```
<CONTROL class="cl-control cl-maskeditor"><TEXT class="cl-text" />
  <CLEAR class="cl-maskeditor-clear" />
</CONTROL>
```

넘버에디터

```
<CONTROL class="cl-control cl-numbereditor"><TEXT class="cl-text" />
  <CLEAR class="cl-numbereditor-clear" />
  <BUTTONS class="cl-numbereditor-buttons">
    <INCREASE class="cl-numbereditor-increase" />
    <DECREASE class="cl-numbereditor-decrease" />
  </BUTTONS>
</CONTROL>
```

데이트인풋

```
<CONTROL class="cl-control cl-dateinput"><TEXT class="cl-text" />
  <CLEAR class="cl-dateinput-clear" /><BUTTON class="cl-dateinput-button" />
  <BUTTONS class="cl-dateinput-spinbuttons">
    <INCREASE class="cl-dateinput-increase" />
    <DECREASE class="cl-dateinput-decrease" />
  </BUTTONS>
</CONTROL>
```

파일인풋

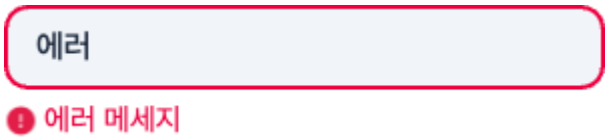
```
<CONTROL class="cl-control cl-fileinput"><TEXT class="cl-text" />
  <CLEAR class="cl-fileinput-clear" />
</CONTROL>
```

ⓘ 테마 파일

input.part.less

사용 가이드라인

- **컨트롤의 너비 및 높이:** 컨트롤의 너비는 데이터의 길이에 맞게 상황에 따라 유동적으로 적용, 높이는 42px 적용



에러 메세지

LESS

```less
// 에러
.is-error {}
```

JS

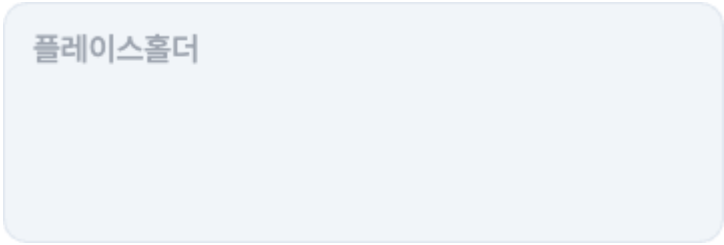
```js
// 에러
Control.style.addClass('is-erorr');
```

웹 접근성 유의사항

- 입력 필드는 aria-label 또는 title 속성을 제공하여 어떤 값을 입력하는지 알 수 있도록 해야 한다.
  - **aria-label:** `fieldLabel` 속성 적용
  - **title:** `tooltip` 속성 적용

텍스트 에리어

여러 줄의 텍스트를 입력할 수 있는 컨트롤



구조

텍스트 에리어 컨트롤의 구조는 다음과 같다.

텍스트에리어

```
<CONTROL class="cl-control cl-textarea">
  <WRAP class="cl-textarea-wrap"><TEXT class="cl-text" /></WRAP>
</CONTROL>
```

ⓘ 테마 파일

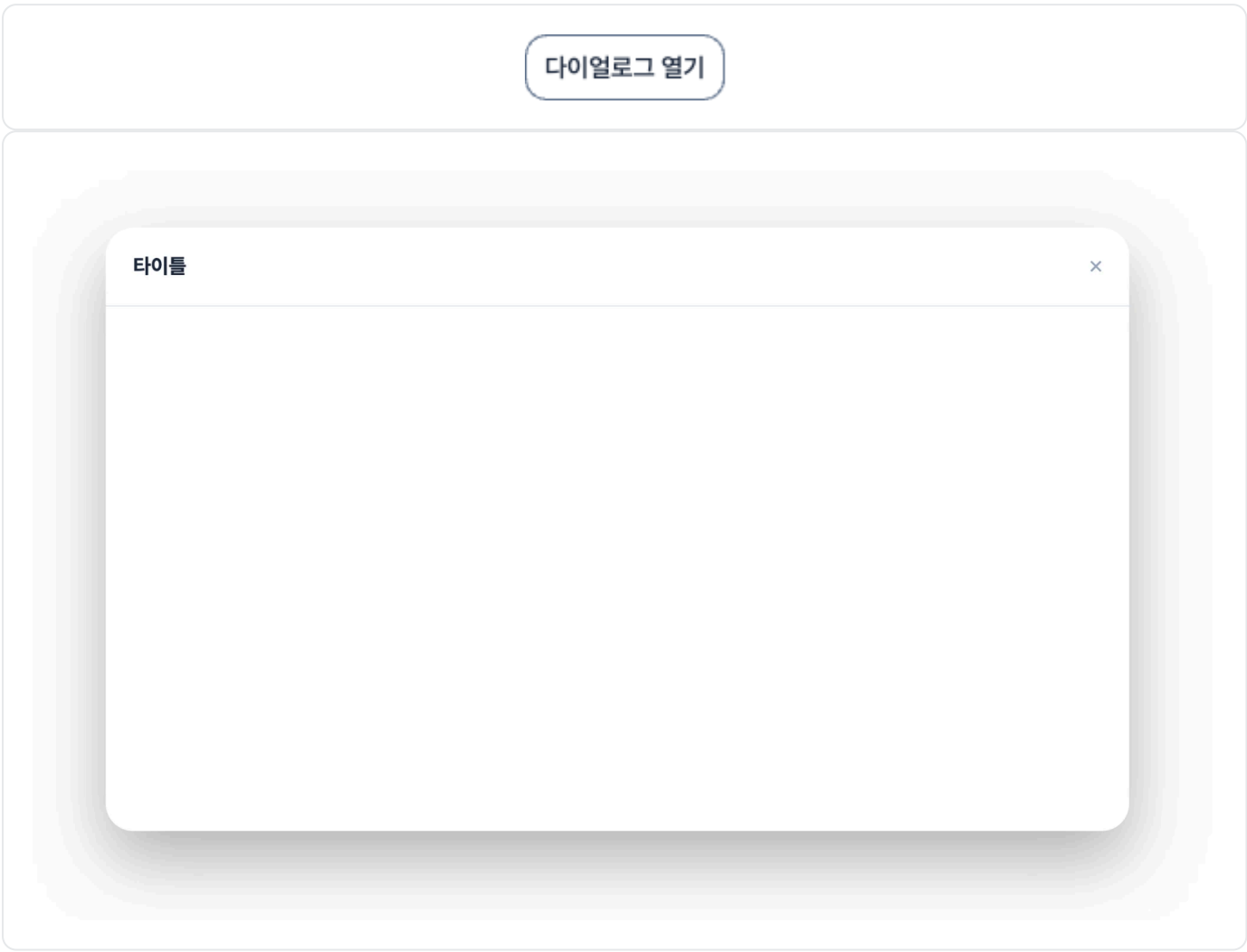
input.part.less

사용 가이드라인

- **글자 수 제한:** maxLength 속성 적용

다이얼로그

대화 상자 컨트롤



JS

```
// 프로젝트 공통 함수 (업무화면에서는 이 방식 사용)
// util.Dialog.open(app, 화면경로, 너비, 높이, 닫힘콜백, 초기값, 옵션)
util.Dialog.open(app, "app/pub/FA/FA011P02", 896, 527, null);

// 루트가 VerticalLayout인 팝업 – 높이 -1이면 내용 높이에 맞춰 자동 조정 (util 전용 기능)
util.Dialog.open(app, "app/pub/FA/FA011P02", 896, -1, null);

// 기본적인 다이얼로그 열기 방식
app.openDialog("앱ID", {
  width: 896,
  height: 527
}).then(function(returnValue) {
  // 다이얼로그가 닫힌 뒤 반환값 처리
});

// 다이얼로그 매니저 열기 방식
app.dialogManager.openDialog("앱ID", "다이얼로그명", {
  width: 896,
  height: 527
}, function(dialog) {
  // 다이얼로그 생성 후 처리
});
```

구조

다이얼로그의 구조는 다음과 같다.

다이얼로그

```
<OVERLAY class="cl-overlay" />
<CONTROL class="cl-control cl-dialog">
  <HEADER class="cl-dialog-header">
    <BUTTON class="cl-dialog-min" />
    <BUTTON class="cl-dialog-max" />
    <BUTTON class="cl-dialog-restore" />
    <BUTTON class="cl-dialog-close" />
  </HEADER>

  <BODY class="cl-dialog-body">
    <CONTROL class="cl-control cl-container">
      ...
    </CONTROL>
  </BODY>
</CONTROL>
```

ⓘ 테마 파일

theme/less/component/dialog.part.less (다이얼로그) · theme/less/custom/layout.part.less (팝업 화면 .pop-content-*)

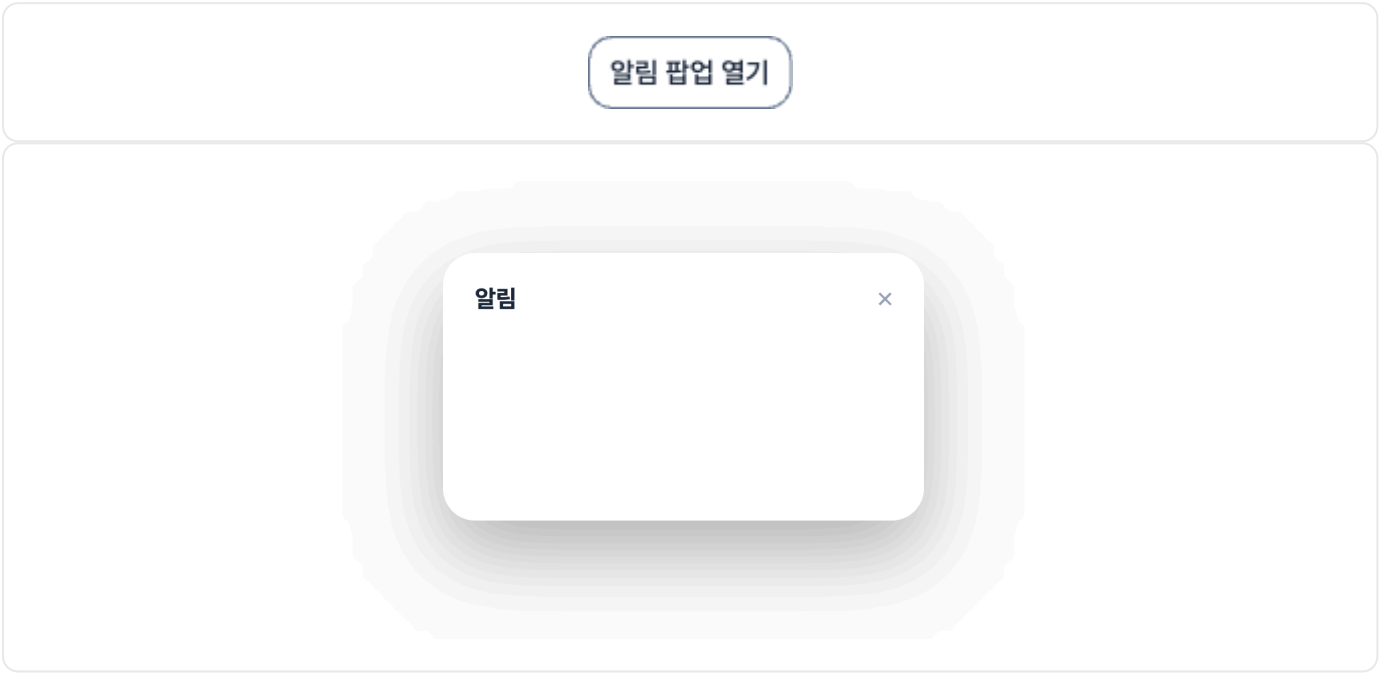
사용 가이드라인

- **다이얼로그 열기:** 다이얼로그 매니저 또는 다이얼로그 API 를 사용하여 열기 (예: openDialog())
- **다이얼로그 크기:** 기본 크기는 896 × 527(너비 × 높이) 한 가지이며, 팝업 내용에 따라 변동할 수 있다

- 너비 896px — 팝업 템플릿(KSPO-POP 스크린) 폭 기준. 내용이 넓은 팝업만 조정
- 높이 527px — 조회팝업 템플릿 기준. 팝업 루트가 FormLayout이면 화면 작성 후 실제 높이로 조정
- 높이 자동(-1) — 팝업 루트가 VerticalLayout이면 높이에 `-1`을 넣어 내용 높이에 맞춘다. 이 기능은 eXBuilder 기본 API(openDialog)가 아니라 공통 모듈 util.Dialog.open()(extension.module.js DialogKit)이 제공하므로 반드시 util로 연다

확장

알림 팝업



JS

```
Dialog.style.addClass('modal');
```

팝업 콘텐츠

팝업이 되는 CLX의 루트 컨테이너(body)에 `.pop-content-wrapper` 클래스를 적용하고, 그 안의 헤더·바디·푸터 그룹에 각각 `.pop-content-header` / `.pop-content-body` / `.pop-content-footer`를 지정한다. 좌우 여백(24px)·영역 간격·푸터 상단 구분선은 테마가 처리하므로 컨테이너에 여백을 직접 넣지 않는다.

LESS

```less
// 팝업 CLX 루트(body)
.cl-container.pop-content-wrapper {
  .cl-container.pop-content-header {
    // 헤더 영역 (조회조건 등)
  }

  .cl-container.pop-content-body {
    // 바디 영역 (그리드·폼 등 본문)
  }

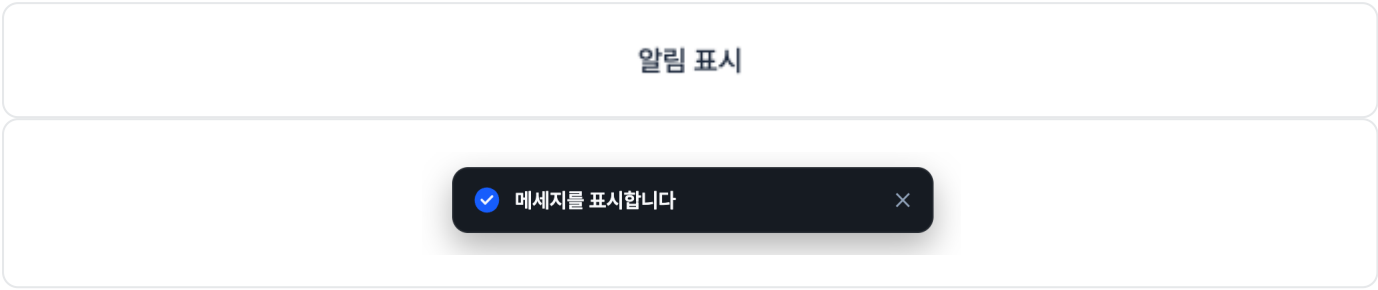
  .cl-container.pop-content-footer {
    // 푸터 영역 (닫기 등 버튼, footer-button-group 버튼은 h40)
  }
}
```

웹 접근성 유의사항

- 다이얼로그가 열릴 때 포커스 진입 위치 설정을 위해 `autoFocusedTarget = 'auto'` 로 설정
- 다이얼로그를 닫을 때 열기 전 위치로 복원하기 위해 `restoreFocus = true` 로 설정

알림

알림을 표현하는 컨트롤



API

```
Notifier.notify('메세지를 표시합니다.');
```

NotificationCenter

```
// 메시지 수신
cpr.core.NotificationCenter.INSTANCE.subscribe('app-msg', this, function(data) {
  var notifier = app.lookup('notifier');
  var message = data.msg;

  if (data.success == true) {
    notifier.success(message);
  } else if (data.info == true) {
    notifier.info(message);
  } else if (data.warning == true) {
    notifier.warning(message);
  } else if (data.danger == true) {
    notifier.danger(message);
  } else {
    notifier.info(message);
  }
});

// 메시지 전파
cpr.core.NotificationCenter.INSTANCE.post('app-msg', {
  msg: '메세지를 표시합니다',
  info: true
});
```

구조

알림의 구조는 다음과 같다.

알림

```
<CONTROL class="cl-control cl-notifier">
  <ITEM class="cl-notifier-item cl-notifier-상태">
    <ICON class="cl-notifier-icon" />
    <TEXT class="cl-text" />
    <BUTTON class="cl-notifier-closebutton" />
  </ITEM>
</CONTROL>
```

ⓘ 테마 파일

notifier.part.less

사용 가이드라인

- **알림의 종류:** default, success, info, warning, danger
- **알림 최대 개수:** maxNotifyCount 로 설정
- API 또는 NotificationCenter 통해 알림 표시 필요

웹 접근성 유의사항

- 자동으로 닫히지 않도록 `delay = 2,147,483,647` 로 설정 (Int 형식의 최댓값)
- 닫기 버튼을 항상 표시하도록 `close = true` 로 설정
- 컨트롤에 키보드 접근 및 복귀 가능 하도록 설정
  - 알림 컨트롤이 띄워진 경우, 현재 포커스 컨트롤에서 탭 이동 시 알림 컨트롤로 이동해야 하고, 알림 컨트롤의 리스트가 모두 닫히는 경우 이전 포커스 컨트롤로 이동해야 한다.
  - `focusItem(item, restoreFocusControl)` API 사용

아웃풋

입력을 할 수 없으며 출력 기능만을 가진 컨트롤

레이블

구조

아웃풋 컨트롤의 구조는 다음과 같다.

아웃풋

```
<CONTROL class="cl-control cl-output"><TEXT class="cl-text" /></CONTROL>
```

ⓘ 테마 파일

output.part.less

사용 가이드라인

- **디스플레이 표현식:** 표시되는 텍스트를 다양한 값으로 혼합하여 보여주어야 하는 경우 displayExp 속성 적용

CLX

```
// 예시 1번 (원 단위 기호 표시)
displayExp => text + "원"

// 예시 2번 (강조 텍스트 표시)
displayExp => sstr(text, ["className1", "className2"])

// 예시 3번 (스크린 리더 전용 텍스트)
displayExp => sstr(text, ["cl-sound-only"])
```

JS

```
// 예시 1번 (원 단위 기호 표시)
Output.displayExp = 'text + "원"';

// 예시 2번 (강조 텍스트 표시)
Output.displayExp = 'sstr(text, ["className1", "className2"])';

// 예시 3번 (스크린 리더 전용 텍스트)
Output.displayExp = 'sstr(text, ["cl-sound-only"])';
```

웹 접근성 유의사항

- 값 변경 사항에 대해 알릴 수 있어야 하는 경우 `ariaLive` 속성 적용
- 헤딩 텍스트를 표시해야 하는 경우 `outputType` 속성 적용

페이지 인덱서

대용량 데이터가 보이는 컨트롤에서 총 데이터양과 관계없이 화면에 출력하고자 하는 데이터양만큼만 보이도록 도와주는 컨트롤



구조

페이지 인덱서의 구조는 다음과 같다.

버튼형

```
<CONTROL class="cl-control cl-pageindexer">
  <BUTTON class="cl-pageindexer-first" />
  <BUTTON class="cl-pageindexer-prev" />
  <AREA class="cl-pageindexer-index-area">
  <BUTTON class="cl-pageindexer-index">
    <WRAP class="cl-text-wrapper">
      <TEXT class="cl-text" />
    </WRAP>
  </BUTTON>
  </AREA>
  <BUTTON class="cl-pageindexer-next" />
  <BUTTON class="cl-pageindexer-last" />
</CONTROL>
```

텍스트형

```
<CONTROL class="cl-control cl-pageindexer">
  <BUTTON class="cl-pageindexer-first" />
  <BUTTON class="cl-pageindexer-prev" />
  <AREA class="cl-pageindexer-index-area">
  <TEXT class="cl-pageindexer-text">
    <WRAP class="cl-text-wrapper">
      <TEXT class="cl-text" />
    </WRAP>
  </TEXT>
  </AREA>
  <BUTTON class="cl-pageindexer-next" />
  <BUTTON class="cl-pageindexer-last" />
</CONTROL>
```

ⓘ 테마 파일

pagination.part.less

사용 가이드라인

- **표시할 버튼 개수:** `viewPageCount` 로 설정 (텍스트형을 사용하는 경우 무시됨)

콤보 박스

미리 정해진 값들 중에서 단일 값을 선택하는 컨트롤



구조

콤보 박스 및 링크드 콤보박스 컨트롤의 구조는 다음과 같다.

콤보박스

```
// 콤보박스
<CONTROL class="cl-control cl-combobox">
  <TEXT class="text" />
  <BUTTON class="cl-combobox-button" />
</CONTROL>

// 콤보박스 리스트
<LIST class="cl-combobox-list cl-popup">
  <WRAP class="cl-viewport">
    <ITEM class="cl-combobox-item">
      <CHECKBOX class="cl-checkbox" />
      <TEXT class="cl-text" />
    </ITEM>
  </WRAP>
</LIST>
```

링크드 콤보박스

```
// 링크드 콤보박스
<CONTROL class="cl-control cl-linkedcombobox">
  <COMBO class="cl-linkedcombobox-combo>
               <TEXT class=" cl-text" />
  <BUTTON class="cl-linkedcombobox-button" />
  </COMBO>
</CONTROL>

// 링크드 콤보박스 리스트
<LIST class="cl-linkedcombobox-list cl-popup">
  <WRAP class="cl-viewport">
    <ITEM class="cl-linkedcombobox-item">
      <TEXT class="cl-text" />
      <ARROW class="cl-linkedcombobox-arrow">
    </ITEM>
  </WRAP>
</LIST>
```

ⓘ 테마 파일

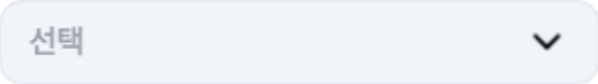
select-box.part.less

사용 가이드라인

- **컨트롤의 너비 및 높이:** 컨트롤의 너비는 데이터의 길이에 맞게 상황에 따라 유동적으로 적용, 높이는 24px 적용
- **검색어 입력 사용:** `preventInput = false` 적용
- **트리 형태 표시:** `dropDownType = 'tree'` 적용

확장

멀티형



상태



LESS

```less
// 에러
.is-error {}
```

JS

```js
// 에러
ComboBox.style.addClass('is-erorr');
```

웹 접근성 유의사항

- 셀렉트 필드는 aria-label 또는 title 속성을 제공하여 어떤 용도인지 확인할 수 있도록 해야 한다.
  - **aria-label:** `fieldLabel` 속성 적용
  - **title:** `tooltip` 속성 적용

체크박스

사용자가 여러 개의 옵션 중 한 개 이상의 값을 선택할 수 있도록 하는 컨트롤



구조

체크박스 및 체크박스그룹의 구조는 다음과 같다.

체크박스

```
<CONTROL class="cl-control cl-checkbox">
  <FIELD class="cl-checkbox-field">
    <WRAP class="cl-icon-wrapper">
      <ICON class="cl-checkbox-icon" />
    </WRAP>
    <TEXT class="cl-text" />
  </FIELD>
</CONTROL>
```

체크박스그룹

```
<CONTROL class="cl-control cl-checkboxgroup">
  <ITEM class="cl-checkbox">
    <FIELD class="cl-checkbox-field">
      <WRAP class="cl-icon-wrapper">
        <ICON class="cl-checkbox-icon" />
      </WRAP>
      <TEXT class="cl-text" />
    </FIELD>
  </ITEM>
</CONTROL>
```

ⓘ 테마 파일

selection.part.less

사용 가이드라인

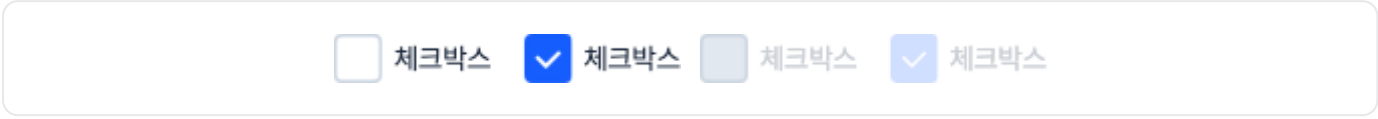
- **컨트롤의 너비 및 높이:** 컨트롤의 너비는 데이터의 길이에 맞게 상황에 따라 유동적으로 적용, 높이는 최소 24px적용
- **아이콘 표시:** `hideIcon` 속성 설정
- **아이콘의 방향:** `iconAlign` 속성으로 적용
- **아이템 너비 고정:** `fixedWidth = false` 으로 적용
- **아이템 선택 영역 설정:** `clickRegion` 속성으로 적용

확장

체크박스그룹

`horizontalSpacing = 16`, `verticalSpacing = 8` 로 설정 (기본값 설정되어 있으므로 별도 처리하지 않아도 됨)

`colCount` 속성으로 한 줄에 표시될 아이템의 개수 지정 (기본값: -1)



라디오 버튼

여러 가지 선택 사항 중 한 가지 만 선택하는 컨트롤



구조

라디오 버튼의 구조는 다음과 같다.

라디오 버튼

```
<CONTROL class="cl-control cl-radiobutton">
  <ITEM class="cl-radiobutton-item">
    <FIELD class="cl-radiobutton-field">
      <WRAP class="cl-icon-wrapper">
        <ICON class="cl-radiobutton-icon" />
      </WRAP>
      <TEXT class="cl-text" />
    </FIELD>
  </ITEM>
</CONTROL>
```

ⓘ 테마 파일

selection.part.less

사용 가이드라인

- **컨트롤의 너비 및 높이:** 컨트롤의 너비는 데이터의 길이에 맞게 상황에 따라 유동적으로 적용, 높이는 최소 36px적용
- **아이템 간격 설정:** `horizontalSpacing = 16`, `verticalSpacing = 8` 로 적용 (기본값 설정되어 있으므로 별도 처리하지 않아도 됨)
- **표시될 아이템 개수 지정:** `colCount` 속성 설정 (기본값: -1)
- **아이콘 표시:** `hideIcon` 속성 설정
- **아이콘의 방향:** `iconAlign` 속성으로 적용
- **아이템 너비 고정:** `fixedWidth = false` 으로 적용
- **아이템 선택 영역 설정:** `clickRegion` 속성으로 적용

토글 스위치

토글은 상호 배타적인 두 가지 상태를 전환하는 데 사용되는 요소



구조

스위치는 체크박스 컨트롤의 구조와 같다.

체크박스

```
<CONTROL class="cl-control cl-checkbox toggle-switch">
  <FIELD class="cl-checkbox-field">
    <WRAP class="cl-icon-wrapper">
      <ICON class="cl-checkbox-icon" />
    </WRAP>
    <TEXT class="cl-text" />
  </FIELD>
</CONTROL>
```

ⓘ 테마 파일

switch.part.less

사용 가이드라인

- **컨트롤의 너비와 높이:** 컨트롤의 너비와 높이는 데이터의 길이에 맞게 상황에 따라 유동적으로 적용하기 위해 자동 크기 관련 속성 지정
- **디스플레이 표현식:** 표시되는 텍스트를 다양한 값으로 혼합하여 보여주어야 하는 경우 displayExp 속성 적용
- **아이템 선택 영역 설정:** `clickRegion = 'item'` 속성으로 적용
- **아이콘의 방향:** iconAlign 속성으로 적용

웹 접근성 유의사항

- 토글 스위치는 aria-label 또는 title 속성을 제공하여 어떤 용도인지 확인할 수 있도록 해야 한다.
  - **aria-label:** `fieldLabel` 속성 적용
  - **title:** `tooltip` 속성 적용

탭 폴더

여러 페이지 중 사용자가 선택한 하나의 페이지를 표시하는 컨트롤



구조

탭 폴더의 구조는 다음과 같다. MDI 폴더는 탭 폴더의 구조를 상속받는다.

탭 폴더

```
<CONTROL class="cl-control cl-tabfolder">
  <HEADER class="cl-tabfolder-header">
    <ITEM class="cl-tabfolder-item">
      <CHECKBOX class="cl-checkbox" />
      <TEXT class="cl-text" />
      <BUTTON class="cl-tabfolder-button" />
    </ITEM>
  </HEADER>

  <BODY class="cl-tabfolder-body">
    <CONTROL class="cl-control cl-container">
      ...
    </CONTROL>
  </BODY>
</CONTROL>
```

MDI 폴더

```
<CONTROL class="cl-control cl-tabfolder cl-mdifolder">
  <HEADER class="cl-tabfolder-header">
    <ITEM class="cl-tabfolder-item">
      <CHECKBOX class="cl-checkbox" />
      <TEXT class="cl-text" />
      <BUTTON class="cl-tabfolder-button" />
    </ITEM>
  </HEADER>

  <BODY class="cl-tabfolder-body">
    <CONTROL class="cl-control cl-container">
      ...
    </CONTROL>
  </BODY>
</CONTROL>
```

ⓘ 테마 파일

tab.part.less

사용 가이드라인

- **아이템 크기:** `itemSizing` 속성 적용 (기본값: 'auto')
- **아이템 간 간격:** `itemSpacing` 속성 적용 (기본값: 0)

웹 접근성 유의사항

- `fieldLabel` 속성으로 탭 폴더의 아이템 개수에 대한 레이블 표시

  CLX

  ```
  fieldLabel => '3개의 탭이 있습니다.'
  ```

  JS

  ```
  Tab.fieldLabel = '3개의 탭이 있습니다.';
  ```

- 키보드 운용에 대한 설명이 제공되어야 하는 경우 `itemDisplayExp` 속성으로 음성 피드백 제공

  CLX

  ```
  itemDisplayExp => 'sstr("좌우 방향키로 탭 페이지를 이동합니다.", ["cl-sound-only"]) + text';
  ```

  JS

  ```
  tabfolder.itemDisplayExp = 'sstr("좌우 방향키로 탭 페이지를 이동합니다.", ["cl-sound-only"]) + text';
  ```

그리드

테이블 형태의 컨트롤



순번	상태	트리셀	기본	아웃풋	인풋박스 *	넘버에디터	서치인풋	마스크에디터	데이트인풋
1		폴더	처리	처리	강남구	1,000	부서1	990909-2**	2023-0
2		서브 폴더	미처리	미처리	성북구	10,000	부서2	950905-2**	2022-1
3		파일 (	처리	처리	동작구	100,000	부서3	940421-1**	2023-0
4		파일 (	미처리	미처리	관악구	1,000,000	부서3	900905-1**	2023-0
5		서브 폴더	미처리	미처리	종로구	10,000,000	부서1	881212-1***	2021-0
6		파일 (	미처리	미처리	서초구	100,000,000	부서2	920411-1***	2017-0
7		파일 (	처리	처리	중랑구	999,999,99	부서1	671212-2***	2019-1
합계					1,000,100,999				

구조

그리드 컨트롤의 구조는 다음과 같다.

그리드

```
<CONTROL class="cl-control cl-grid">
  <SPLIT class="cl-grid-centersplit">
    <HEADER class="cl-grid-header" />
    <DETAIL class="cl-grid-detail" />
    <FOOTER class="cl-grid-footer" />
  </SPLIT>
</CONTROL>
```

그리드 헤더

```
<CONTROL class="cl-control cl-grid">
  <SPLIT class="cl-grid-centersplit">
    <HEADER class="cl-grid-header">
      <WRAP class="cl-grid-cell-inherit">
        <ROW class="cl-grid-row">
          <CELL class="cl-grid-cell cl-grid-cell-inherit">
            <WRAP class="cl-gird-cell-inherit">
              <WRAP class="cl-gird-cell-inherit">
                <CONTROL class="cl-control cl-checkbox cl-builtin cl-grid-ctrl-inherit" />
              </WRAP>
            </WRAP>
          </CELL>
          <CELL class="cl-grid-cell cl-grid-cell-inherit">
            <WRAP class="cl-gird-cell-inherit">
              <WRAP class="cl-gird-cell-inherit">
                <CONTROL class="cl-control cl-default-cell cl-grid-cell-inherit" />
              </WRAP>
            </WRAP>
          </CELL>
          <CELL class="cl-grid-cell cl-grid-cell-inherit">
            <BUTTON class="cl-filter" />
            <WRAP class="cl-gird-cell-inherit">
              <WRAP class="cl-gird-cell-inherit">
                <CONTROL class="cl-control cl-default-cell cl-grid-cell-inherit" />
              </WRAP>
            </WRAP>
          </CELL>
          <CELL class="cl-grid-cell cl-grid-cell-inherit">
            <WRAP class="cl-gird-cell-inherit cl-sortable-cell">
              <BUTTON class="cl-sort cl-sort-asc">
                <ICON class="cl-sort-icon" />
                <TEXT class="cl-sort-seq" />
              </BUTTON>
              <WRAP class="cl-gird-cell-inherit">
                <CONTROL class="cl-control cl-default-cell cl-grid-cell-inherit" />
              </WRAP>
```

그리드 디테일

```
<CONTROL class="cl-control cl-grid">
  <SPLIT class="cl-grid-centersplit">
    <DETAIL class="cl-grid-detail">
      <ROW class="cl-grid-row">
        <CELL class="cl-grid-cell cl-grid-cell-inherit">
          <CONTROL class="cl-control cl-checkbox cl-builtin cl-grid-ctrl-inherit" />
        </CELL>
        <CELL class="cl-grid-cell cl-grid-cell-inherit">
          <CONTROL class="cl-control cl-default-cell cl-rowindex cl-builtin" />
        </CELL>
        <CELL class="cl-grid-cell cl-grid-cell-inherit">
          <CONTROL class="cl-control cl-inputbox cl-grid-ctrl-inherit" />
        </CELL>
        ...
      </ROW>
    </DETAIL>
  </SPLIT>
</CONTROL>
```

그리드 푸터

```
<CONTROL class="cl-control cl-grid">
  <SPLIT class="cl-grid-centersplit">
    <FOOTER class="cl-grid-footer">
      <WRAP class="cl-grid-cell-inherit">
        <CELL class="cl-grid-cell cl-grid-cell-inherit">
          <WRAP class="cl-grid-cell-inherit">
            <CONTROL class="cl-control cl-default-cell cl-grid-ctrl-inherit" />
          </WRAP>
          ...
        </CELL>
      </WRAP>
    </FOOTER>
  </SPLIT>
</CONTROL>
```

사용 가이드라인

루트

- **트리셀 또는 그룹핑 접고 펼치기:** `collapsible = true`
- **컬럼 너비 자동조정 여부:** `autoFit`
  - [필수] 모든 특수컬럼의 경우 컬럼 너비 자동조정 미사용 설정
  - [필수] 버튼, 체크박스, 라디오 버튼 컨트롤의 경우 컬럼 너비 자동조정 미사용 설정
  - [옵션] UI 또는 데이터의 규격이 일정하게 유지되는 항목(날짜, 코드값 등)은 컬럼 너비 자동조정 미사용 설정 가능
- **컬럼 이동 가능 여부:** `columnMovable = true`
- **컬럼 너비 조정 여부:** `resizableColumns = 'all'`

특수 컬럼

- **특수 컬럼의 사용:** `columnType` 속성 설정
- **특수 컬럼 너비:** `60px` 기본, 헤더가 텍스트가 아니라 체크박스·컨트롤이면 `52px`

특수 컬럼	너비	autoFit	텍스트
checkbox	52px | 60px	false	`columnType = 'checkbox'` | 선택
radio	52px | 60px	false	`columnType = 'checkbox'` | 선택
rowindex	60px	false	순번

헤더 셀

- **행 높이:** `40px`
  - [2줄 헤더] rowspan으로 셀을 병합해 2줄로 구성하는 경우, 각 행(gridrow)을 모두 40px로 설정
- **다중 정렬 사용:** `sortable = true` (`targetColumnName` 속성으로 대상 컬럼 설정)
- **데이터 필터 사용:** `filterable = true` (`targetColumnName` 속성으로 대상 컬럼 설정)
- **필수입력 컬럼 표시:** `required` 클래스 사용

디테일 셀

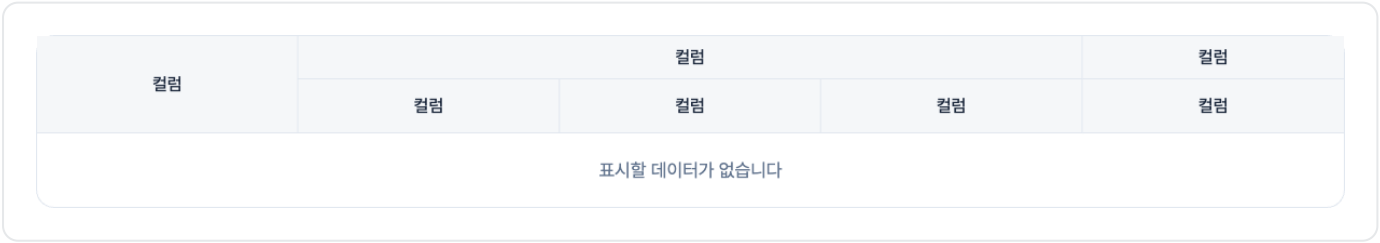
- **행 높이:** `50px`
- **컨트롤의 사용:** 데이터 타입별 최적 컨트롤 배치
  - 텍스트 입력: 인풋박스, 서치인풋, 텍스트에리어
  - 선택 입력: 콤보박스, 체크박스, 라디오 버튼
  - 숫자 입력: 넘버에디터
  - 날짜 입력: 데이트인풋
- **컨트롤의 정렬 설정:** 텍스트는 좌측, 숫자는 우측, 데이터의 규격이 일정하게 유지되는 항목 값은 중앙 정렬을 기본으로 함
- **필수입력 컬럼 표시:** `required` 클래스 사용

푸터 셀

- **행 높이:** `50px`
- **컨트롤의 사용:** 아웃풋 컨트롤 배치

확장

컬럼 두줄



데이터 없음



CLX

```
noDataMessage => '표시할 데이터가 없습니다.'
```

웹 접근성 유의사항

- 그리드는 aria-label 또는 title 속성을 제공하여 표시되는 목록에 대한 설명을 작성해야 한다.
  - **aria-label:** `fieldLabel` 속성 적용
  - **title:** `tooltip` 속성 적용
- 웹 접근성 준수 페이지의 경우 그리드를 읽기 전용(`readOnly = true`)으로 작성 권장
  - 값 변경이 필요한 경우 그리드 행의 값이 바인딩된 그룹에서 편집 가능하도록 디자인 필요
  - 그리드 헤더에서 필터 또는 정렬 기능을 사용하는 경우 그리드 외부에서 이를 조작할 수 있도록 제공 필요
  - 그리드 영역에서 자동으로 가상커서가 해제되어야하나 일부 스크린리더에서 가상커서가 자동으로 해제되지 않는 문제가 있어 해당 내용을 대체텍스트 형태로 제공하는 것을 권장 (예: 그리드 이전에 접근하는 아웃풋 컨트롤을 생성하여 "다음에 접근하는 그리드는 스크린리더 사용시 가상커서를 해제하고 사용해야 합니다."와 같은 문구 추가

툴팁

요소나 본문 텍스트에 제공되는 짧은 부가 설명으로, 설명이 필요한 대상 또는 별도의 활성화 버튼에 마우스를 올렸을 때 설명 텍스트 표시



구조

툴팁 컨트롤의 구조는 다음과 같다.

툴팁

```
<CONTROL class="cl-tooltip">
  <IN class="cl-control">
    <TEXT class="cl-text" />
  </IN>
</CONTROL>
```

ⓘ 테마 파일

tooltip.part.less

사용 가이드라인

- **툴팁 매니저의 사용:** 툴팁 매니저 및 컨트롤의 tooltip 속성이 설정되어야 함

CLX

```
tooltip => 툴팁 텍스트
```

JS

```
// 특정 컨트롤에 툴팁 매니저 적용
cpr.core.Platform.INSTANCE.tooltipManager.registerTooltip(app.lookup("컨트롤ID"), function(tooltipOwner) {
  var output = new cpr.controls.Output();
  output.value = tooltipOwner.tooltip;
  return output;
});

// 특정 컨트롤에 연결된 툴팁 제거
cpr.core.Platform.INSTANCE.tooltipManager.unregisterTooltip(app.lookup("컨트롤ID"));

// 모든 컨트롤에 툴팁 매니저 적용
cpr.core.Platform.INSTANCE.tooltipManager.setDefaultTooltip(function(tooltipOwner) {
  var output = new cpr.controls.Output();
  output.value = tooltipOwner.tooltip;
  return output;
});
```

웹 접근성 유의사항

- 툴팁 매니저 사용 시 aria-describedby 속성 자동 설정

트리

트리 형태의 데이터를 표현하는 컨트롤



구조

트리의 구조는 다음과 같다.

트리

```
<CONTROL class="cl-control cl-tree">
  <WRAP class="cl-tree-wrap">
    <WRAP class="cl-viewport">
      <LIST class="cl-tree-list">
        <WRAP class="cl-subtree">
          <ITEM class="cl-tree-item">
            <ICON class="cl-tree-treeicon" />
            <ICON class="cl-tree-treeitemicon" />
            <TEXT class="cl-text" />
            <BADGE class="cl-badge" />
          </ITEM>
        </WRAP>
      </LIST>
    </WRAP>
  </WRAP>
</CONTROL>
```

트리셀

```
<CONTROL class="cl-control cl-treecell cl-grid-ctrl-inherit">
  <ITEM class="cl-treecell-item">
    <ICON class="cl-treecell-treeicon" />
    <ICON class="cl-treecell-itemicon" />
    <TEXT class="cl-text" />
  </ITEM>
</CONTROL>
```

ⓘ 테마 파일

tree.part.less

사용 가이드라인

- **아이템 들여쓰기 간격:** 20

확장

체크박스 사용

showItemCheckbox 또는 showSelectionCheckbox 속성으로 체크박스를 표시



> **showItemCheckbox 와 showSelectionCheckbox 속성의 차이점은?**
>
> showItemCheckbox 는 아이템 체크상태 표시 용도의 체크박스 표시 여부를 설정하며, showSelectionCheckbox 는 아이템 선택상태 표시 용도의 체크박스 표시 여부를 설정한다.

웹 접근성 유의사항

- 트리는 aria-label 또는 title 속성을 제공하여 어떤 값을 선택하는지 알 수 있도록 해야 한다.
  - **aria-label:** `fieldLabel` 속성 적용
  - **title:** `tooltip` 속성 적용

타이포그래피

텍스트의 크기·굵기·색상·정렬을 지정하는 공통 클래스



클래스	크기
`.fs-xl`	17px
`.fs-lg`	16px
`.fs-ml`	15px
`.fs-md`	14px
`.fs-xs`	13px
`.fs-xxs`	12px

> ⓘ .fs-ml은 md와 lg 사이
>
> `.fs-ml` (15px)은 medium-large의 약자로 `.fs-md` (14px)와 `.fs-lg` (16px) 사이 단계다. 이름 순서와 크기 순서가 같으므로 위 표 순서대로 고르면 된다.

구조

타이포그래피 클래스는 두 종류다. 크기·굵기·색상·정렬처럼 한 가지 속성만 바꾸는 유틸리티 클래스(.fs-* / .fw-* / .text-* 등)와, 캡션·컨트롤 라벨·링크처럼 여러 속성을 묶어 역할을 표현하는 역할 클래스(.caption / .ctrl-label / .link)가 있다. 아웃풋의 class 속성에 그대로 붙여 쓴다.

> ⓘ 테마 파일
>
> theme/less/common/globals.part.less (유틸리티 클래스) · theme/less/custom/typography.part.less (역할 클래스)

사용 가이드라인

- **기본 크기:** 아웃풋·버튼 등 컨트롤의 기본 글자 크기는 14px, 인풋·콤보박스 입력값과 그리드 헤더는 16px, 그리드 셀과 폼(form-base) 내부는 17px, 섹션 타이틀은 18px이다. 기본 크기 그대로 쓸 때는 크기 클래스를 붙이지 않는다
- **크기·굵기 지정:** 크기는 .fs-*, 굵기는 .fw-* 클래스로 지정하고, 디자인 뷰에서 인라인 스타일(font-size, font-weight)을 직접 입력하지 않는다

- **타이틀:** 화면 타이틀과 그리드·폼 타이틀은 `udcComAppHeader` / `udcComGridTitle` / `udcComFormTitle`가 자체 스타일을 가지고 있으므로 타이포 클래스를 덧붙이지 않는다
- **지시문:** 화면 상단 안내 문구는 텍스트 클래스로 꾸미지 않고 guide-box(강조 안내 영역)를 사용한다
- **더 굵은 강조:** fw-bold(700)보다 굵은 강조가 필요하면 클래스를 새로 추가해서 쓴다

확장

굵기

4단계 굵기 클래스. 업무화면에서는 강조 값(합계·상태·점수 등)에 .fw-bold를 가장 많이 쓴다.

Aa 가나다라 · .fw-bold · 700

Aa 가나다라 · .fw-semibold · 600

Aa 가나다라 · .fw-medium · 500

Aa 가나다라 · .fw-normal · 400

텍스트 색상

의미에 따라 색상 클래스를 선택한다. .text-secondary와 .text-subtle은 같은 색이다.

Aa 가나다라 · .text-primary — 기본 텍스트

Aa 가나다라 · .text-secondary — 보조 텍스트

Aa 가나다라 · .text-subtlest — 가장 옅은 보조 텍스트

Aa 가나다라 · .text-disabled — 비활성 (#A7AFBA)

Aa 가나다라 · .text-point — 포인트

Aa 가나다라 · .text-success — 성공·정상

Aa 가나다라 · .text-info — 정보

Aa 가나다라 · .text-warning — 주의

Aa 가나다라 · .text-danger — 오류·위험

> ⓘ **비활성 색상**
>
> 비활성 텍스트는 `#A7AFBA`이다(디자인의 사용불가 인풋 — 배경 #E2E8F0 위 기본 텍스트 30%와 같은 색). 인풋·콤보박스·그리드·탭 등 컨트롤에 enabled=false를 주면 테마가 자동으로 이 색을 적용하므로 클래스를 따로 붙이지 않는다. `.text-disabled`는 컨트롤 상태와 상관없이 텍스트만 비활성처럼 보이게 할 때 쓴다.

이 밖에 .text-red / .text-blue / .text-gray, 회색 단계 .text-gray-10 ~ .text-gray-95도 사용할 수 있다.

캡션

단위 표시, 건수 요약, 입력 보조 설명처럼 본문보다 한 단계 약한 보조 텍스트에 쓴다. 14px · 옅은 회색으로 표시된다.

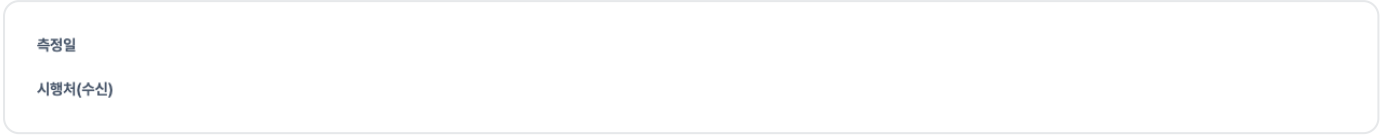
(단위: 백만원)

조회 3 / 합격 1 / 체크 0

※ 숫자 입력 시 자동으로 평균이 계산됩니다.

컨트롤 라벨

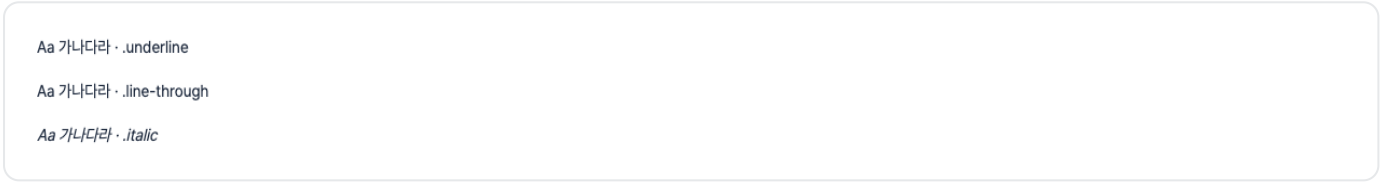
`.ctrl-label` — 폼레이아웃의 label 셀이 아닌 곳(조회 영역 한 줄 배치, 콘텐츠 조건 영역 등)에서 컨트롤 앞에 붙는 짧은 라벨에 쓴다. 아웃풋의 class에 ctrl-label을 지정하며, 14px · semibold · 조회 라벨 색으로 표시된다.



링크

클릭해서 이동하거나 팝업을 여는 텍스트에 쓴다. 대표 색상으로 표시되고, 마우스를 올리면 밑줄이 생긴다.

장식·정렬·줄바꿈



클래스	설명
`text-left / text-center / text-right`	텍스트 정렬. 그리드·폼 셀 안의 아웃풋 정렬에 쓴다(숫자·금액은 text-right)
`truncate`	한 줄로 표시하고 넘치는 부분은 말줄임(...) 처리
`whitespace-nowrap`	줄바꿈 없이 한 줄로 표시
`break-all`	긴 영문·숫자도 글자 단위로 줄바꿈
`no-underline`	밑줄 제거

숫자 너비 고정

`.fv-numeric` — 숫자 폭을 같게 맞춰(tabular-nums) 세로로 나열된 숫자의 자릿수가 어긋나지 않게 한다.



- 금액 · 건수 · 일시처럼 위아래로 비교하는 숫자(그리드 숫자 컬럼, 합계, 요약 카드 수치 등)에 쓴다.
- 실시간으로 바뀌는 숫자(카운트 · 타이머)에 쓰면 값이 바뀔 때 글자 폭이 흔들리지 않는다.
- 정렬(text-right)은 따로 지정한다. fv-numeric은 숫자 폭만 맞춘다.

자주 쓰는 조합

실제 업무화면에서 반복되는 조합이다.

클래스	용도
`fw-bold`	합계·점수·상태 등 강조할 값
`caption`	단위, 건수 요약, 입력 보조 설명
`text-gray-50`	대기·미처리처럼 흐리게 보여줄 상태 값
`text-red`	확정 연도·경고 수치 등 눈에 띄어야 하는 값 (배지와 조합: sub-badge text-red)
`text-right`	그리드·폼 안의 숫자·금액

웹 접근성 유의사항

- **최소 크기:** 본문 텍스트는 14px 미만으로 줄이지 않는다
- **색상 대비:** `.text-subtlest`, `.text-gray-10 ~ 40` 등 옅은 색상은 배경과의 명도 대비(WCAG AA 4.5:1)를 확인한 후 사용한다
- **색상만으로 구분 금지:** 오류·경고처럼 의미를 전달하는 텍스트는 색상만으로 구분하지 말고 문구(예: '불일치', '오류')를 함께 표시한다

화면 패턴 개요

화면에서 반복적으로 쓰이는 CRUD 처리 패턴 정리. 화면 설계 시 참고용이며 상황에 따라 유동적으로 조합해서 쓴다.

패턴 종류

처리 대상 데이터의 구성 · 관계에 따른 기본 패턴과, 화면 안에서 반복적으로 쓰이는 기능 패턴으로 나눈다. 요즘 업무 화면에서 실제로 자주 쓰는 구성 위주로 정리했다.

분류	패턴명	내용	대응 템플릿
기본 패턴	List	목록/그리드 조회 화면. 단독으로 쓰거나 상세 화면의 진입점 역할을 한다	`F_그리드` · `F_그리드_페이징` · `F_그리드_지시문`
기본 패턴	Single (등록/상세)	1건의 데이터를 등록 · 수정 · 조회하는 폼 화면	`V_폼` · `V_폼+폼` · `V_폼+폼+폼` · `V_폼(좌)+폼`
기본 패턴	List + Detail (인라인)	목록에서 선택한 항목의 상세를 같은 화면 안에서 함께 보여주는 구성	`F_그리드+폼` · `F_그리드(좌)+폼` · `V_폼(좌)+그리드` · `V_폼+그리드+그리드`
기본 패턴	List → Detail (팝업/별도 화면)	목록에서 선택 시 팝업이나 별도 화면으로 상세를 여는 구성	`F_그리드` + 팝업화면 템플릿 (`util.Dialog.open`)
기본 패턴	Master-Detail (1:N)	상위 목록 선택 시 하위 목록이 함께 갱신되는 계층형 구성	`F_마스터(좌)+디테일` · `V_마스터+디테일`
기본 패턴	Split (좌우/상하 분할)	화면을 두 영역으로 나눠 서로 다른 콘텐츠를 동시에 보여주는 구성 (목록+목록, 트리+목록 등)	`F_그리드(좌)+그리드` · `F_그리드+그리드` · `V_그리드+그리드` · `F_그리드_셔틀`
기능 패턴	Tab	여러 문서 또는 패널을 하나의 창에 두고 전환하여 볼 수 있도록 하는 화면 패턴	`F_탭` · `F_그리드(좌)+탭`
기능 패턴	Tree	계층 데이터를 탐색 · 선택할 때 사용하는 화면 패턴	전용 템플릿 없음 — 분할 템플릿의 왼쪽 영역에 트리 배치

대응 템플릿은 `templates/일반화면` 기준이며, 팝업은 `templates/팝업화면`을 사용한다.

정보 구분

화면 안의 데이터는 역할에 따라 Main 정보와 Sub 정보(상세 · 종속 · 연관)로 나눠서 배치한다.

요소	설명
`Main 정보`	조회한 결과 또는 화면의 핵심정보, 검색결과가 보여지는 영역.
상세정보	List에서 선택된 1건에 대한 상세정보를 나타내는 영역.
종속정보	메인 정보 1건에 종속된 내용이 구성될 때 사용.
연관정보	메인 정보와 독립된 정보로 구성되지만 밀접한 연관성이 있는 정보.

> ⓘ **Sub 정보 배치 우선순위**
>
> Main 정보를 상(좌) 영역에 배치하고, Sub 정보 중 중요도가 낮은 정보를 하(우) 영역에 배치한다. 우선순위는 ① 상세정보 ② 종속정보 ③ 연관정보 순이다.

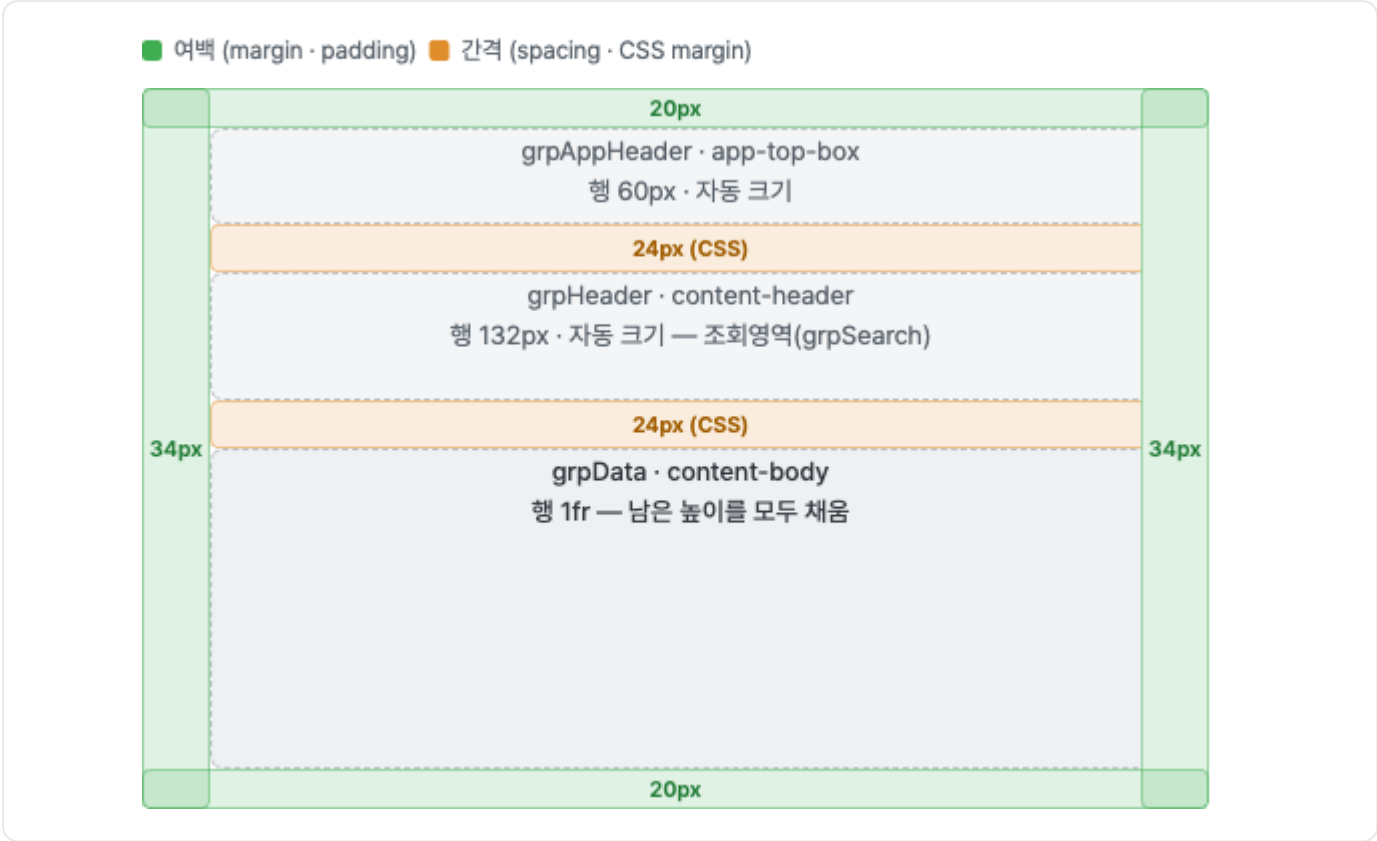
대표 예시

위 10개 패턴의 화면 구성을 간단한 목업으로 표현하면 다음과 같다. 실제로는 패턴별로 여러 배치 변형이 있으며, 여기서는 대표 형태만 보여준다.



최상위 레이아웃

화면 루트부터 앱헤더 · 조회영역 · 바디 · 푸터까지의 골격과 여백 규칙



구조

화면 골격은 **폼레이아웃(F)**이 기본이다. 루트 `body.content-wrapper`를 3행 FormLayout으로 두고 바디 행을 `1fr`로 지정해 그리드가 남은 영역을 채운다. (템플릿 접두어 `F_` 폼레이아웃 / `V_` 버티컬)

- **grpAppHeader (1행)** — 앱 타이틀(udcComAppHeader) + 전체제어 버튼(udcComHeadBtn). 미사용 시 visible=false.
- **grpHeader (2행)** — 조회영역(grpSearch). 지시문(guide-box)이 있으면 그 아래에 둔다.
- **grpData (3행)** — 콘텐츠(.content) 영역. 행 1fr로 남은 높이를 채운다.

F(폼레이아웃) / V(버티컬) 판단 기준

- **F가 기본**이다. 그리드가 남은 높이를 채워야 하면 F로 작성한다.
- 폼 + 그리드 구성은 **높이 720~740 기준 그리드 5행이 보이는지**로 판단하고, 안 나오면 V로 작성한다.
- 그리드를 고정 높이로 나열하거나 폼만 있는 화면은 V를 사용한다.
- 애매하면 디자인 편집기가 아닌 **브라우저 미리보기 높이**로 판단한다.

사용 클래스 · ID

`templates` 화면 템플릿의 그룹 ID 명명 규칙을 필수로 따른다.

ID	클래스	설명
(body)	content-wrapper	일반화면 루트. 배경색만 CSS로 들어가며 여백은 레이아웃 속성으로 지정한다.
grpAppHeader	app-top-box	앱헤더. 아래 24px 간격은 CSS margin-bottom으로 처리된다.
grpHeader	content-header	조회영역 래퍼. VerticalLayout spacing 24.
grpSearch	search-box	조회조건 영역. 배치 규칙은 컨트롤 레이아웃 문서 참고.
grpData	content-body	바디. 내부 배치 규칙은 콘텐츠 레이아웃 문서 참고.

API Reference — 일반화면(F)

`F_` 템플릿의 컨테이너별 기준값.

컨테이너	레이아웃	간격(h / v)	여백	행
body.content-wrapper	FormLayout, scrollable=true	0px / 0px	상하 20px · 좌우 34px	60px 자동 / 132px 자동 / 1fr
grpHeader	VerticalLayout	spacing 24	–	–
grpData	FormLayout, scrollable=false	24px / 24px	–	1fr
.content	FormLayout, scrollable=false	24px / 18px	–	36px 자동 / 1fr

사용 가이드라인

- 영역 사이 24px는 CSS margin-bottom(`app-top-box` · `content-header`)으로 들어가므로 루트 간격은 0으로 둔다.
- 조회영역이 2줄을 넘기면 2행이 132보다 커질 수 있다. (예: `F_그리드_지시문` 228px)
- `grpAppHeader` 를 `visible=false` 로 두면 루트의 해당 행을 **숨김(hidden)** 처리한다.
- 스크롤은 루트에만 두고 나머지 그룹은 `scrollable=false` 로 설정한다.
- 레이아웃을 전환 · 수정한 컨테이너는 **표시순으로 자식 정렬**(`Ctrl+Shift+K` / Mac `⇧⌘K`)을 실행한다. (읽기 · 탭 순서 일치)

자동 크기 사용

- `px` 행(60 / 132 / 36 / 팝업 푸터 72)은 **자동 크기 사용을 반드시 체크**한다.
- 입력한 `px` 는 편집기 표시용이며, 실제로는 내용 높이로 계산된다. (앱헤더 미사용 시 빈칸이 남지 않음)
- `1fr` 행은 해당 없음 — FR 단위에서는 체크박스가 사라진다.
- 기준 행 외에는 사용하지 않는다. (상위 그룹 전체 재계산 유발)

버티컬(V) 화면

그리드를 고정 높이로 나열하거나 폼만 있는 화면. 루트를 VerticalLayout으로 두고 루트에서 스크롤된다.

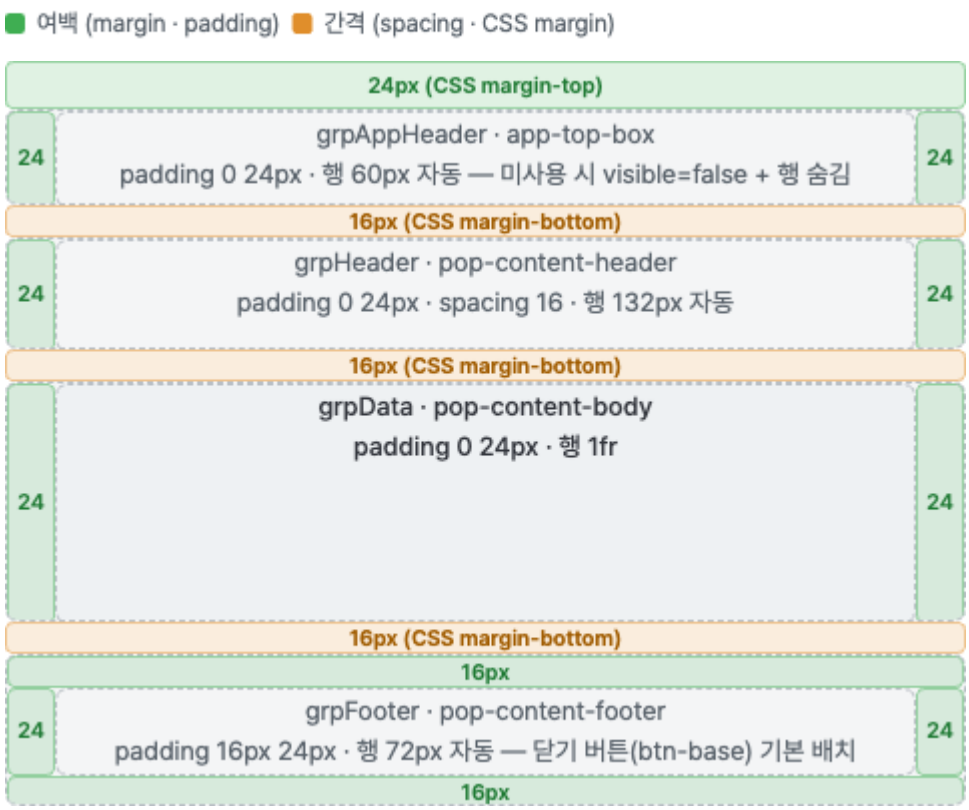


컨테이너	레이아웃	간격	여백
`body.content-wrapper`	`VerticalLayout, scrollable=true`	`spacing 0`	상하 `20` · 좌우 `34` (단위 없이 입력)
`grpData`	`VerticalLayout`	`spacing 24`	–
`.content`	`VerticalLayout`	`spacing 18`	–

- 그리드는 고정 높이(`autosize=none`)를 사용한다 — 1개 배치 `292px` (5행), 폼과 함께 · 여러 개 나열 `192px` (3행).
- 높이 = 헤더 40px + 행 50px × N + 보더 2px.
- **292 · 192px은 기본값이며 화면 구성에 따라 조절할 수 있다.** 노출 행 수를 정해 위 계산식으로 맞춘다. (예: 4행 `242px` , 7행 `392px`)

팝업 화면

일반화면과 같은 폼레이아웃 골격에 푸터(grpFooter) 행이 추가된다. 루트에는 여백을 넣지 않으며, 좌우 24px은 각 영역의 CSS padding(`0 24px` , 푸터 `16px 24px`), 영역 간 16px은 CSS margin-bottom으로 들어간다.



ID	클래스	설명
(body)	pop-content-wrapper	팝업 루트. 여백 없음.
grpAppHeader	app-top-box	타이틀 영역. padding 0 24px · margin 24px(상) 16px(하). 미사용 시 visible=false + 1행 숨김.
grpHeader	pop-content-header	조회영역. VerticalLayout spacing 16 (일반화면 24) · padding 0 24px · margin-bottom 16px.
grpData	pop-content-body	바디. padding 0 24px · margin-bottom 16px. (.content 박스 스타일 제거)
grpFooter	pop-content-footer	푸터. padding 16px 24px + 상단 구분선.
–	footer-button-group	푸터 버튼 그룹. FlowLayout hspacing 12, 우측 정렬.

컨테이너	레이아웃	간격(h / v)	여백	행
body.pop-content-wrapper (F)	FormLayout, scrollable=false	0px / 0px	없음	60px 자동 / 132px 자동 / 1fr / 72px 자동
body.pop-content-wrapper (V)	VerticalLayout, scrollable=true	spacing 0	없음	–
grpFooter	FormLayout	1fr / 0px	–	1fr

- 루트 FormLayout은 `scrollable=false`로 설정한다. (일반화면은 true)

- **푸터는 기획서에 없어도 기본 배치**하고 닫기 버튼(`btn-base`)을 둔다. 숨길 때는 `visible=false` + 루트 4행 숨김.
- 팝업 템플릿 스크린은 `KSPO-POP` (896px).

> ⓘ 팝업은 util.Dialog.open으로 연다
>
> util.Dialog.open(app, "app/pub/<업무폴더>/<화면ID>", 896, 높이, null) — 너비 896 고정, 높이는 루트가 VerticalLayout이면 -1, FormLayout이면 고정값. (미작성 팝업은 임시 527)

임베디드 화면

탭 · 임베디드앱 안에 들어가는 화면(`F_프레임_임베딩`). 바디만 구성한다.



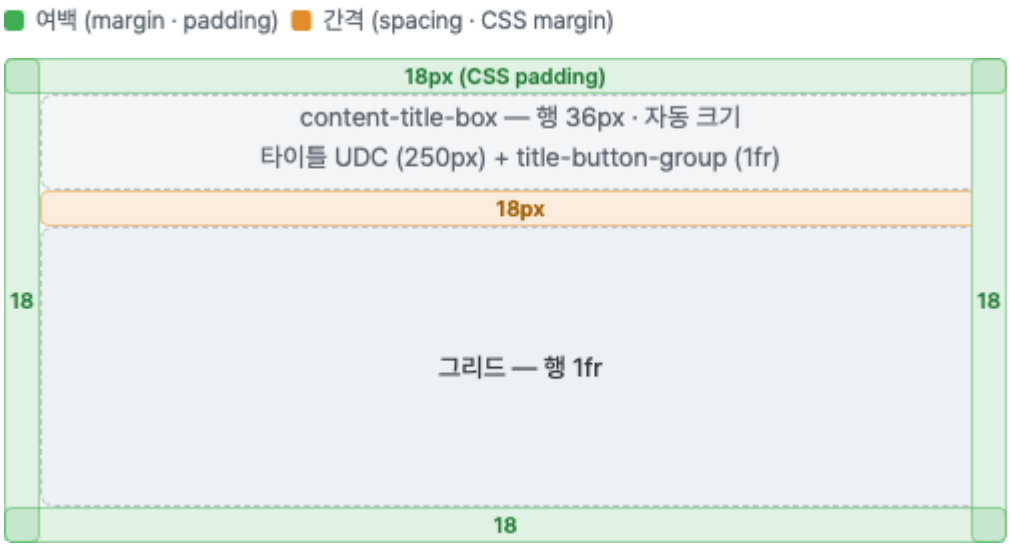
컨테이너	레이아웃	간격(h / v)	여백	행
body.inner-content-wrapper	FormLayout, scrollable=true	24px / 24px	없음	1fr
grpData · inner-content-body	FormLayout, scrollable=false	24px / 24px	–	1fr

- 여백은 부모 화면이 가지므로 넣지 않는다.
- **부모 화면에 맞춘다** — 부모가 F면 F, V면 V(`V_프레임_임베딩`).

> ⓘ 레이아웃 CSS는 직접 지정하지 않는다
>
> display · position · left/top/right/bottom · width · height 등은 eXBuilder6 레이아웃이 관리하는 속성이라, 직접 지정하면 이후 속성을 되돌려도 반영되지 않는 등 부작용이 생긴다.

콘텐츠 레이아웃

바디(grpData) 안의 .content · 타이틀 박스 · 분할 그룹 배치 규칙



구조

콘텐츠 블록의 기본 단위는 `.content` 이며, 타이틀 박스(`content-title-box`)와 그리드 · 폼을 세로로 쌓는다. 배경 · 보더 · 18px padding은 CSS로 들어가므로 레이아웃 여백은 넣지 않는다. (팝업에서는 박스 스타일 제거)

- **content-title-box** — 타이틀 UDC(udcComGridTitle / udcComFormTitle) + 버튼 그룹(title-button-group). 열 250px 자동 / 1fr.
- **그리드 · 폼** — 타이틀 박스 아래 배치. F 화면 그리드는 행 1fr.

라이브 샘플

템플릿 `.content` 구성 그대로 — 그리드 타이틀(건수) + 기능 · 엑셀 · 행 제어 | 구분선 | 서브 · 메인.



사용 클래스

클래스	설명
content-body	바디(grpData). 팝업 pop-content-body, 임베디드 inner-content-body.
content	콘텐츠 블록 단위. (CSS 배경 · 보더 · padding 18px)
content-title-box	.content 상단 타이틀 행. 자식은 CSS로 세로 가운데 정렬된다.
title-button-group	타이틀 오른쪽 버튼 그룹(FlowLayout, halign right).
separator	기능성 · 액션 버튼 사이 구분선. (1px × 30px 아웃풋)
sub-text	타이틀 우측 합계 · 상태 등 보조 텍스트.
caption	그리드 아래 단위 표기 등 캡션(행 36px 자동).
division-group	콘텐츠를 좌우로 나누는 그룹.
master-detail-group	세로 마스터-디테일의 디테일 묶음. (CSS 상단 구분선 · padding 24px)
master-detail-group horizontal	좌우 배치 마스터-디테일의 디테일 묶음(상단 구분선 없음).
selected-target	디테일 타이틀의 "OOO 선택됨" 뱃지. 단독 클래스로 쓴다.
guide-box	지시문 박스.
content-condition-box	콘텐츠 안에서 그리드와 함께 저장하는 입력값 영역.

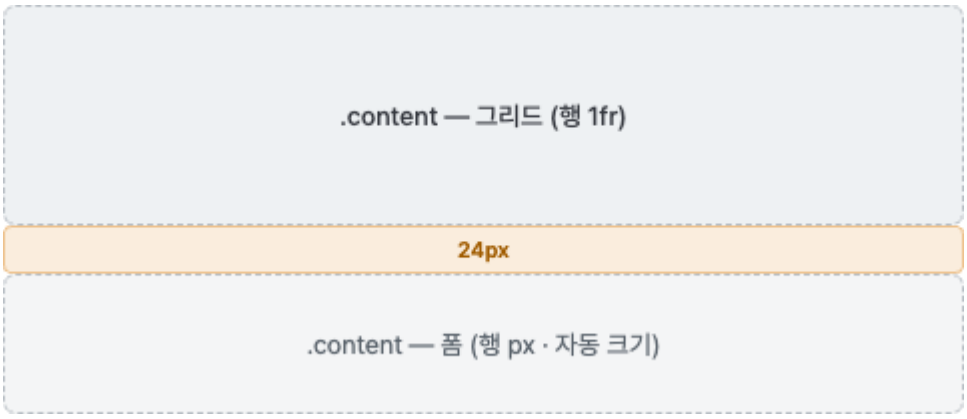
사용 가이드라인

- `.content` 는 **그리드 행만 `1fr`**, 타이틀 박스 · 캡션 등 나머지 행은 `36px 자동` .
- `form-base` 만 있는 `.content` 는 VerticalLayout(spacing 18)을 유지한다.
- `.content` 는 간격 `18` 로 만든 뒤 `horizontalSpacing` 만 `24px` 로 수정한다. (대화상자 간격은 가로 · 세로 동일 적용)
- 버튼은 기능(`btn-function`) / 기본(`btn-base`) / 중요(`btn-primary`) 순으로 배치하고, 기능성 · 액션 버튼 사이에는 `separator` 를 넣는다.
- 행 제어는 `udcComGridCudBtns` 를 사용하며, 행추가 · 행삭제가 있으면 행취소 포함 3개 노출이 기본이다.
- 타이틀이 없으면 UDC는 `visible=false` 로 숨기고, 미사용 **버튼은 삭제**한다.
- 푸터에 총계 건수만 있으면 `udcComGridTitle` 의 `rowCount` · `rowCountVisible=true` 를 사용한다.

예시

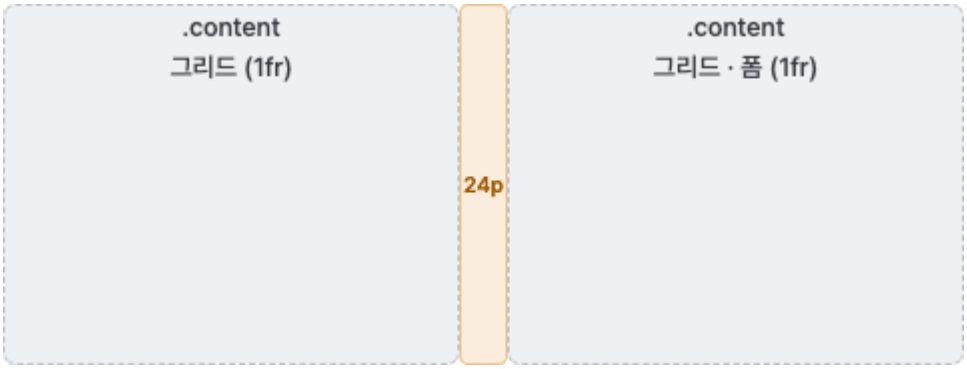
세로 배치

`F_그리드+그리드` · `F_그리드+폼` . `grpData` (24px / 24px)에 `.content` 를 행마다 배치한다. (그리드+그리드 `1fr` + `1fr` , 그리드+폼 `1fr` +폼 높이 px 자동)



좌우 배치 (division-group)

`F_그리드(좌)+그리드` · `F_그리드(좌)+폼` . `division-group` (FormLayout 24px / 24px)의 열을 `1fr` + `1fr` , 행을 `1fr` 로 두면 좌우 높이가 맞춰진다. 각 `.content` 도 FormLayout.



- 반응형 설정값은 소스에 넣어 둔다 — `division-group` column-count `1` , `.content` min-height `382` (tablet · mobile).

마스터-디테일

마스터 행 선택 시 디테일 영역이 갱신되는 패턴. (좌우 `F_마스터(좌)+디테일` , 세로 `V_마스터+디테일`)

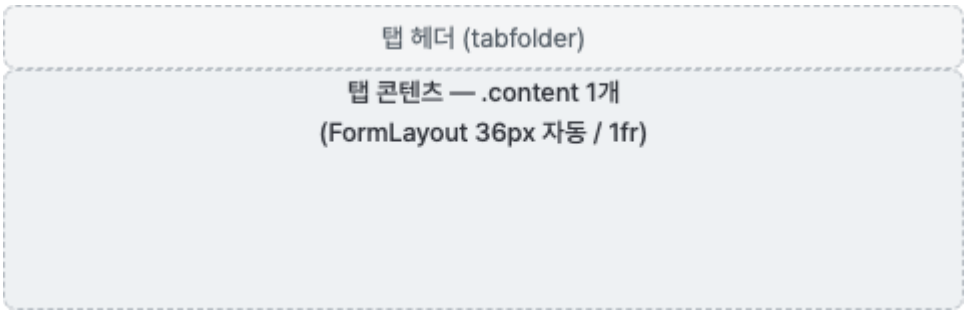


division-group (1fr + 1fr) › 왼쪽 .content | 오른쪽 master-detail-group horizontal

- 디테일 묶음은 좌우 `master-detail-group horizontal` (10px / 10px, 행 `1fr` / `390px` 자동), 세로 `master-detail-group` (spacing 10).
- 디테일 타이틀에 `selected-target` 뱃지를 둔다. 마스터-디테일이거나 기획서에 "OOO 선택됨"이 있을 때만, `udcFileUpload` 디테일은 제외.

탭

`F_탭` · `F_그리드(좌)+탭` . `grpData`에 탭폴더를 두고 탭마다 `.content`를 하나씩 배치한다.



지시문 (guide-box)

기본 위치는 `grpHeader` 안 조회영역 아래(`F_그리드_지시문`). 기획서상 특정 콘텐츠에 강조돼 있으면 `.content` 타이틀 박스 아래도 가능하다. 빨간 안내문도 지시문으로 작성한다.



컨텐츠 조건 설정 박스 (content-condition-box)

콘텐츠 안에서 전체 값(이상유무 · 출근인원 등)을 입력해 아래 그리드와 함께 저장하는 영역. 조회조건이 아니므로 `search-box`를 재사용하지 않는다.

API Reference

콘텐츠 영역 컨테이너별 기준값.

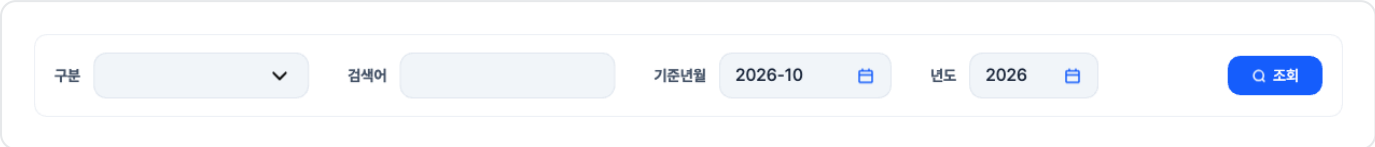
컨테이너	레이아웃	간격(h / v)	행 · 열
grpData (F)	FormLayout, scrollable=false	24px / 24px	행 1fr (콘텐츠 수만큼)
.content – 그리드 (F)	FormLayout, scrollable=false	24px / 18px	행 36px 자동 / 1fr (/ 36px 자동 – 캡션)
.content – 폼만	VerticalLayout	spacing 18	–
content-title-box	FormLayout	기본값	열 250px 자동 / 1fr
title-button-group	FlowLayout, halign=right	기본값	–
division-group	FormLayout	24px / 24px	열 1fr / 1fr · 행 1fr
master-detail-group horizontal	FormLayout, scrollable=false	10px / 10px	행 1fr / 390px 자동
master-detail-group (V)	VerticalLayout	spacing 10	–
grpData (V)	VerticalLayout	spacing 24	–
.content (V)	VerticalLayout	spacing 18	그리드 고정 292px · 192px (화면 구성에 따라 조절)

ⓘ 그리드 높이는 F와 V가 다르다

F는 그리드 행 1fr(높이 고정 안 함), V는 고정 높이(기본 292 · 192px, 화면 구성에 따라 조절 가능). 2줄 헤더(rowspan)는 행마다 40px.

컨트롤 레이아웃

조회영역(search-box) · 입력 폼(form-base)의 라벨 · 컨트롤 배치 규칙



구조

`label` 아웃풋 + 컨트롤 한 쌍이 기본 단위이며, 두 가지 형태로 나뉜다.

- **조회영역 (search-box)** — 라벨 · 컨트롤을 FlowLayout으로 배치, 오른쪽 88px 열에 조회 버튼(udcSearchBtn).
- **입력 폼 (form-base)** — 라벨 열 160px + 컨트롤 열 1fr을 반복(2단 · 3단)하는 FormLayout.

사용 클래스

클래스	설명
search-box	조회조건 영역(grpSearch). search-ctrl(1fr) + sch-btn-wrap(88px).
search-ctrl	조회 라벨 · 컨트롤을 담는 FlowLayout 그룹.
sch-btn-wrap	조회 버튼(udcSearchBtn) 영역.
label	라벨 아웃풋. (조회영역: 굵게 / form-base: 음영 배경 · 가운데 정렬)
label required	필수 항목 라벨. 별표(*)가 CSS로 자동으로 붙는다.
required (컨트롤)	필수 입력 컨트롤. util.validate 필수 검사 대상.
form-base	입력 · 상세 폼 컨테이너(보더 · 라운드 · 행 구분선).
form-base tbl	테이블형 폼. 세로 구분선이 켜지고 라벨 · 출력이 가운데 정렬된다.
form-control	한 셀에 컨트롤을 2개 이상 두는 묶음(인풋 + 버튼, 인풋 + 체크박스 등).
btn-inline	form-control 안 인풋 옆 버튼.
text-right	숫자 아웃풋 우측 정렬.

조회영역 (search-box)

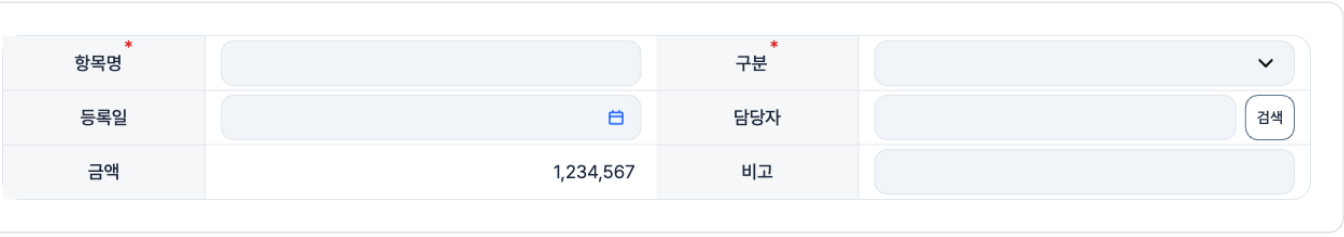
위 샘플이 템플릿 조회영역 구성이다. 필수 항목은 컨트롤에 `required` 클래스를 지정한다.

- 컨트롤 높이 `42px`, 조회 버튼 `88px × 36px` (udcSearchBtn).
- 인풋박스는 기본 `200px`를 사용하며 기획서 입력값이 짧으면 `160px` 등으로 줄인다. (연-월: `160px`, 연도: `120px` 등)
- 라벨은 `autosize=width`, 컨트롤은 `allowNewLine=false`로 설정한다. (줄바꿈 시 라벨 분리 방지)
- 기간 조회는 `udcFromToDate` — 앱 속성 `defaultFieldLabel`만 입력하면 “ 시작일” / “ 종료일”이 자동으로 붙는다.
- `fieldLabel` 등 접근성 속성을 누락하지 않는다.

속성	적용값	내용
search-box columns	1fr 자동 / 88px 자동	search-ctrl · sch-btn-wrap 2열, 행은 1fr 자동
search-ctrl hspacing / vspacing	24 / 16	FlowLayout 간격 (라벨-컨트롤 사이는 CSS로 좁혀짐)
sch-btn-wrap	halign=right, hspacing 12	조회 버튼 FlowLayout
컨트롤 높이	42px	인풋박스 · 콤보박스 · 데이트인풋 · 서치인풋 공통
컨트롤 너비	200px (기본)	입력값이 짧으면 160px 등, 연-월 160px · 연도 120px
allowNewLine	false	모든 컨트롤 flowlayoutdata에 지정

입력 폼 (form-base)

필드를 표 형태로 나열하는 등록 · 상세 폼. 라벨 `160px` + 컨트롤 `1fr` 을 2단(4열) · 3단(6열)으로 반복한다.



- 라벨 열은 기본 `160px` 이며, 라벨 길이나 화면 구성에 따라 가변적으로 조절할 수 있다. (예: `120px` · `200px`)
- 행 높이 `42px` (여러 줄 컨트롤 `90px`).
- 라벨 셀은 `ignore-layout-spacing=true` , 사용자 속성 `needs-auto-height=true` 를 지정한다.
- 한 셀에 컨트롤이 2개 이상이면 `form-control` (열 `1fr` + px 자동)로 묶는다. (인풋 옆 버튼 `btn-inline`)
- 숫자 아웃풋은 `text-right` .
- 반응형 설정값은 템플릿 그대로 유지 — `tablet-column-count=4` , `mobile-column-count=2` .

속성	적용값	내용
columns	160px (기본, 가변) / 1fr (반복)	라벨 열 · 컨트롤 열. 2단은 4열, 3단은 6열. 라벨 열 폭은 화면에 맞게 조절 가능
rows	42px (여러 줄 90px)	필드 행 높이
margins	상하 4px · 좌우 28px	form-base 안쪽 여백. tbl 변형도 동일
horizontalSpacing / verticalSpacing	28px / 8px	열 사이 · 행 사이 간격
hseparatortype / hseparatorwidth	BY_CLASS / 1	행 사이 구분선
ignore-layout-spacing (라벨 셀)	true	라벨 셀이 여백 · 간격 없이 셀을 꽉 채우도록 지정

! 라벨에 별표를 직접 입력하지 않는다

별표(*)는 label required가 CSS로 붙인다. 필수 검사가 필요한 컨트롤에는 required 클래스를 따로 지정한다.